KCC 2026 TUTORIAL

Inference workload, KVCache and LMCache

How a KV cache layer plugs into vLLM — from inference fundamentals, through the architecture and data paths, down to the module internals.

Xcena, yitae jeong

© 2026 XCENA Inc.

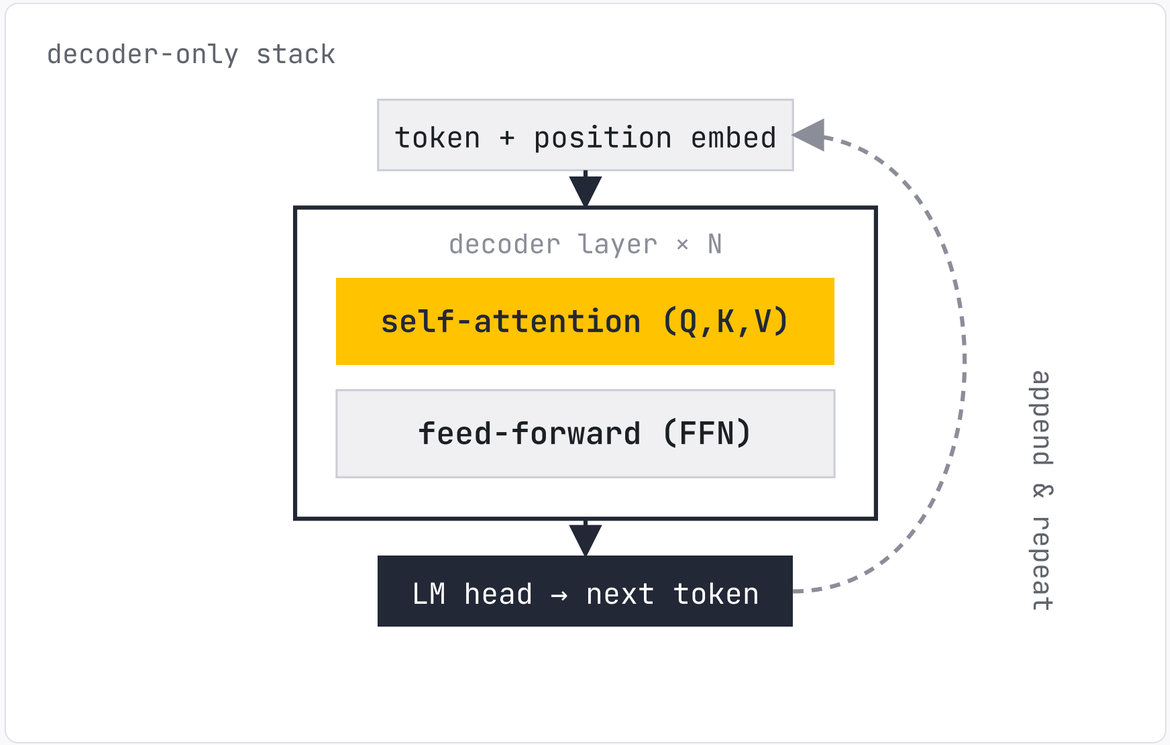
PART 1

Inference Workload

How a transformer turns tokens into KV, why attention shapes the cache, and the workloads that drive reuse.

© 2026 XCENA Inc.

Decoder-Only Transformer, One Token at a Time



Decoder-only

N identical layers — each = self-attention + FFN. No separate encoder.

Autoregressive

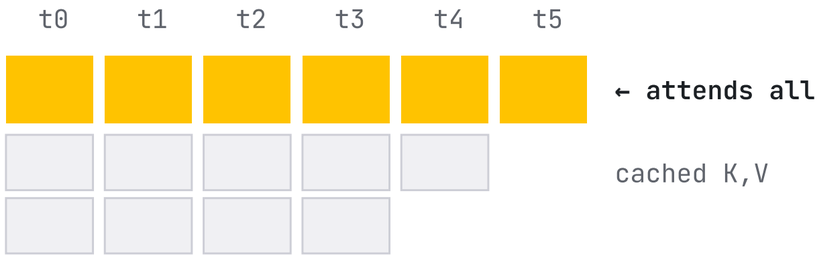
one forward pass → one token → append → repeat.

Every new token attends to all prior tokens → **the reason to cache K,V.**

© 2026 XCENA Inc.

Attention Produces a KV Cache

causal self-attention · token t attends to 0…t



$$attn(t) = softmax(q \cdot K^T) \cdot V$$

K,V depend only on the past

→ compute once, cache, reuse every later step

PER TOKEN

$$Q = xW_Q \cdot K = xW_K \cdot V = xW_V$$

A past token's K,V never change

they depend only on tokens before them — stable forever.

So we cache them

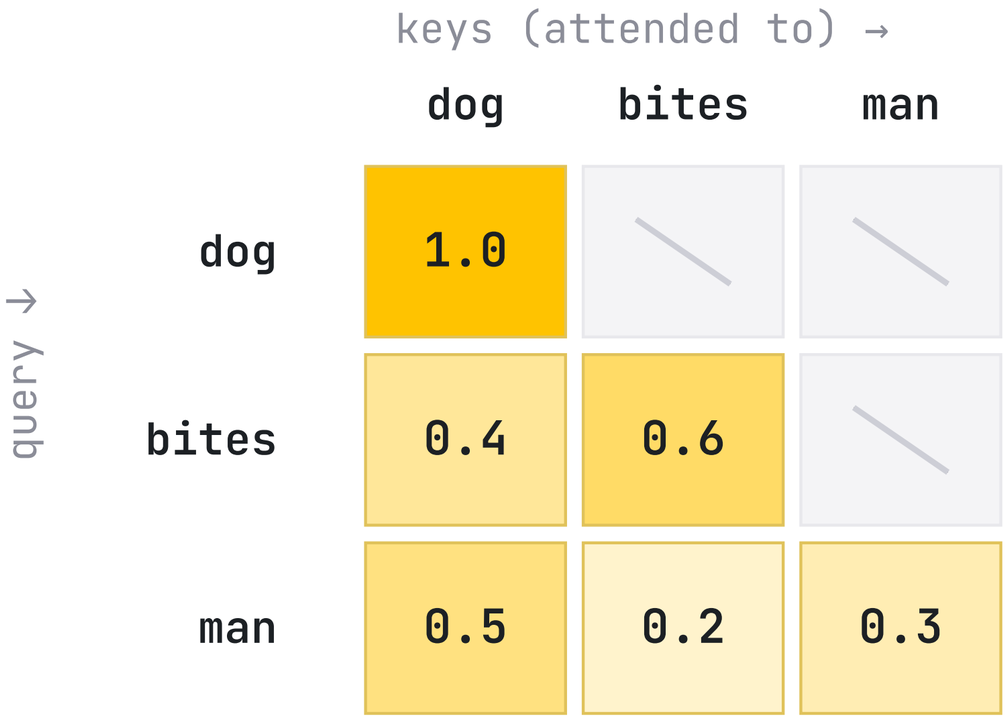
that stored table of K,V **is the KV cache**.

© 2026 XCENA Inc.

Same Tokens, Different Order → Different KV

"dog bites man"

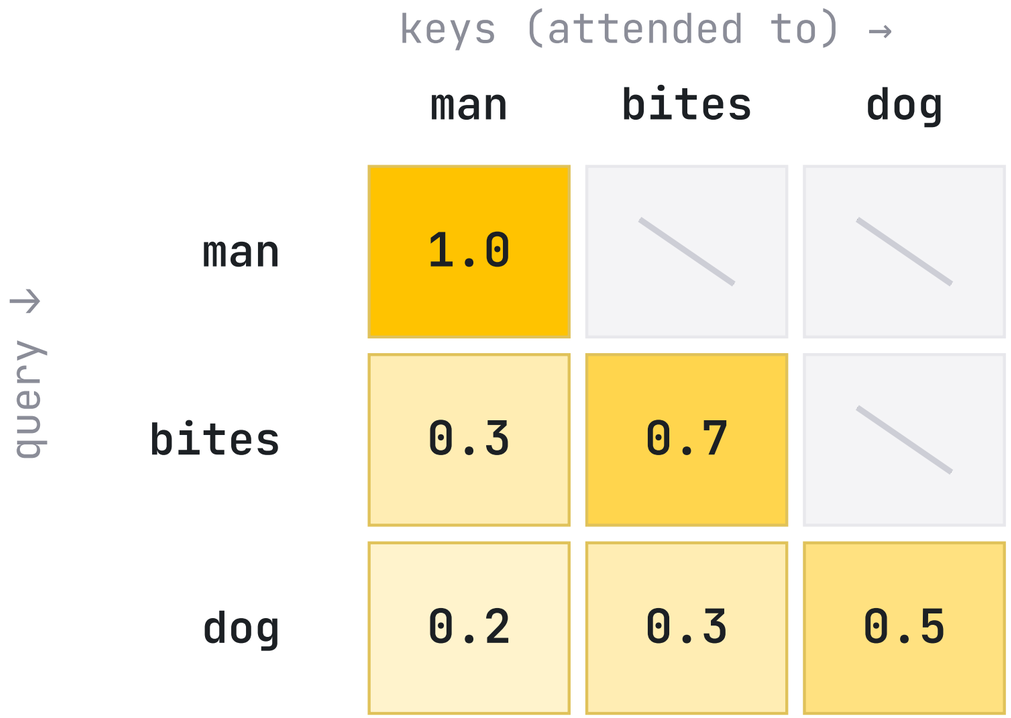
dog$_0$ | bites$_1$ | man$_2$



dog at position **0** → k(dog@0)

"man bites dog"

man$_0$ | bites$_1$ | dog$_2$



dog at position **2** → k(dog@2) **≠ k(dog@0)**

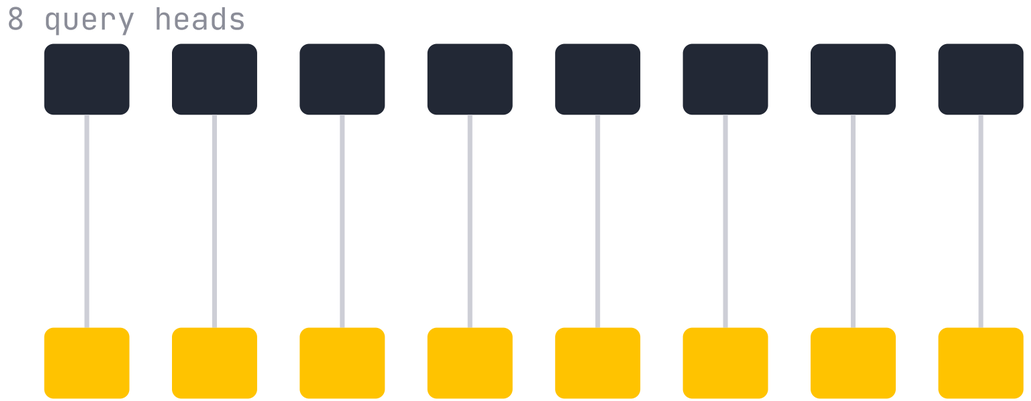
K,V depend on **position + the whole prefix** → cached KV is reusable only on an **exact prefix match**.

© 2026 XCENA Inc.

MHA → GQA: Fewer K,V Heads, Smaller Cache

MHA

Multi-Head Attention



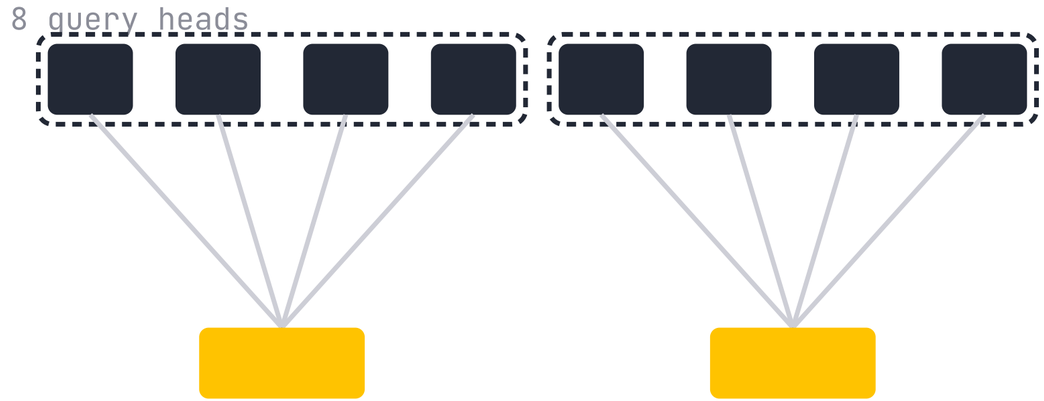
8 K,V heads — one per query head

+ Highest quality — full head capacity
+ Simple, universally supported
– Largest KV cache
– Memory-bound, slower decode

~512 KB/tok · Llama-2-7B

GQA

Grouped-Query Attention



2 K,V heads — each shared by a group of 4

+ ~4× smaller cache, ~same quality
– Slight quality drop vs MHA

~128 KB/tok · Llama-3-8B

KV footprint is set by the **number of K,V heads** — GQA shares them across query heads to shrink the cache.

© 2026 XCENA Inc.

INFERENCE · KV CACHE FOOTPRINT

How Big Is the KV Cache?

KV cache per token =

2	×	head_dim	×	n_kv_heads	×	layers	×	bytes
K and V		d = 128		GQA		model depth		fp16 = 2

worked example · Llama-3-8B (fp16)

2 × 128 × 8 × 32 × 2 B = **128 KB / token** → × 128K tokens = **~16 GB**

K&V × head_dim × KV heads × layers × bytes. Halve the KV heads → halve the cache. That is the whole lever GQA pulls.

MODEL	ATTENTION	KV / TOKEN	@ 128K CTX
Qwen2.5-7B	GQA · 4 KV heads	56 KB	**~7 GB**
Llama-3-8B	GQA · 8 KV heads	128 KB	**~16 GB**
Llama-3-70B	GQA · 8 KV heads	**320 KB**	**~40 GB**

KV heads = K,V heads shared across the 32 query heads (fewer → smaller cache). Llama-3-70B's 40 GB @128K = **half an 80 GB H100** — for a single request.

© 2026 XCENA Inc.

Weights + KV Cache Share One HBM

H100 · 80 GB HBM · Llama-3-8B fp16

~60 GB ÷ 128 KB/tok ≈ **480K tokens** of KV

Llama-3-8B → 16 GB weights fits 1× H100

Llama-3-70B → 140 GB weights needs ≥ 2× H100

Weights are fixed
they claim HBM first; bigger models leave less for KV.

KV pool is the bottleneck
shared by all requests; caps batch size & context length.

Pool full → **evict + recompute**, or **offload to a bigger tier.**

© 2026 XCENA Inc.

Prefill Builds It, Decode Reuses It

PREFILL · WHOLE PROMPT AT ONCE

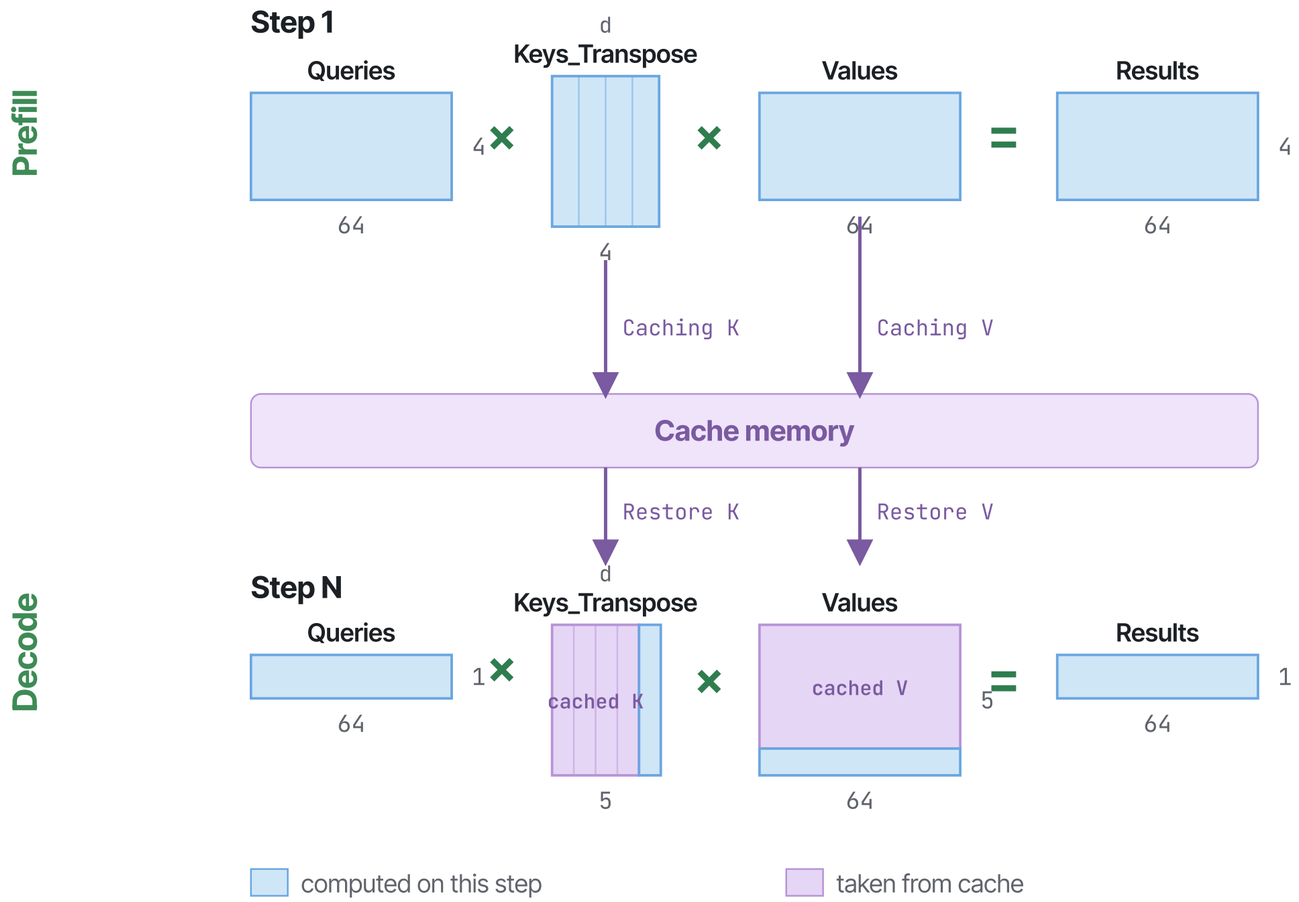
The cat sat on the

5 tokens → K,V computed & cached in 1 pass → 1st output

DECODE · ONE NEW TOKEN / STEP

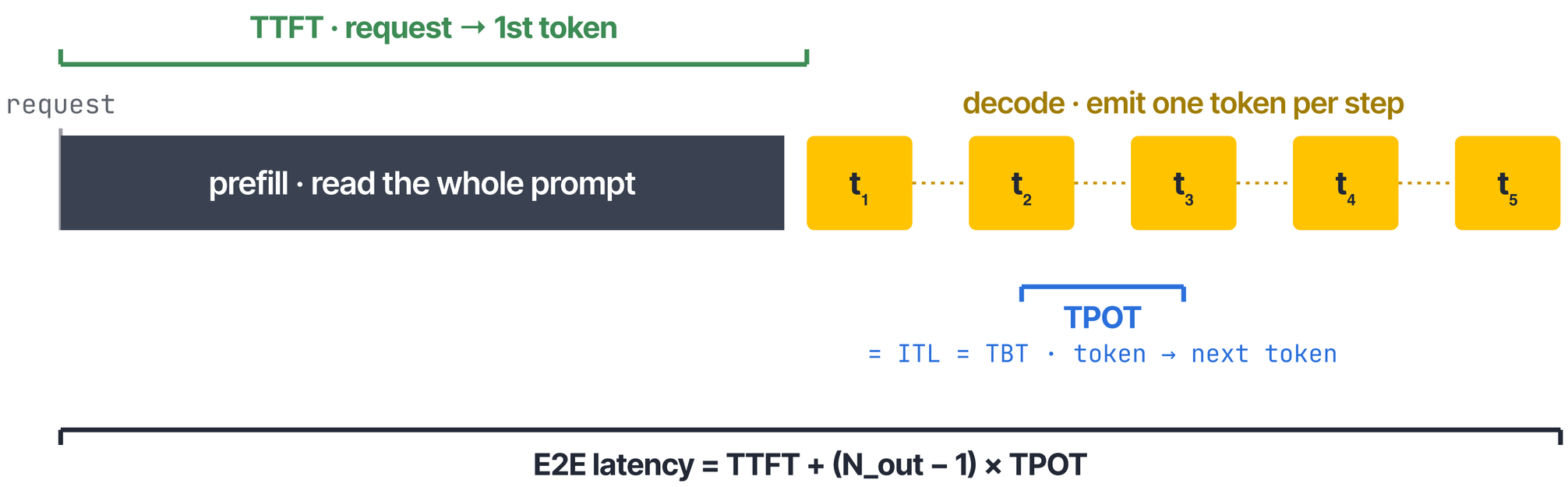
The cat sat on the → mat

reuse 5 cached K,V + compute only the new token's K,V



© 2026 XCENA Inc.

TTFT, TPOT, and Throughput



PREFILL **TTFT**

Time to first token — request to the first generated token. Dominated by prefill.

`lever · cut prefill: KV reuse, chunked prefill`

DECODE **TPOT / ITL**

Time between two consecutive output tokens. Same number as **ITL** (inter-token latency) and **TBT** (time-between-tokens).

`lever · batch size, memory bandwidth, kernels`

BOTH **E2E latency**

Whole-request time: TTFT + (N_out – 1) × TPOT.

`lever · lower both phases`

SYSTEM **Throughput**

Tokens/sec & requests/sec across all concurrent requests.

`lever · batching — trades some latency`

Reuse cuts prefill, and prefill **is** TTFT — so KV reuse is fundamentally a **TTFT** story.

© 2026 XCENA Inc.

INFERENCE · THE CORE TRADEOFF

Reuse or Recompute? A Cost Race

OPTION A Recompute

run prefill again on the GPU

`prefix tokens` → `GPU prefill` → `K,V`

COST

compute-bound

≈ prefix_tokens × FLOPs ÷ GPU

Always available, no cache needed — but grows with prefix length.

OPTION B Reuse

fetch stored K,V from cache memory

`cache tier` → `fetch / copy` → `K,V`

COST

I/O-bound

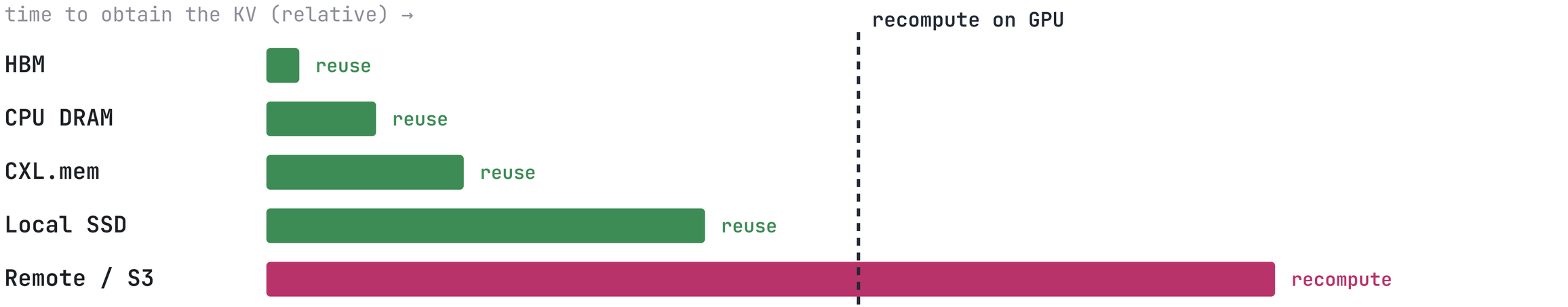
≈ KV_bytes ÷ tier_BW + latency

Near-zero compute — but speed depends on which tier holds it.

Decision: `t_fetch < t_recompute` → reuse the cached KV, otherwise recompute it.

The race, across cache tiers

illustrative — bar = fetch time, line = recompute cost for a given prefix



© 2026 XCENA Inc.

INFERENCE · WORKLOADS THAT DRIVE REUSE

Four Shapes of KV Reuse

Long-document QA / RAG

one doc, many questions — reuse the document's KV.

Multi-turn chat

prefix grows each turn — reuse the whole history.

Agentic / tool use

long prompt + tools repeat across many steps.

Long system prompt · API

same instructions lead every request.

One common thread: a **long, shared prefix** — cheaper to load than to recompute.

© 2026 XCENA Inc.

Agentic AI: Three Axes of Behavior

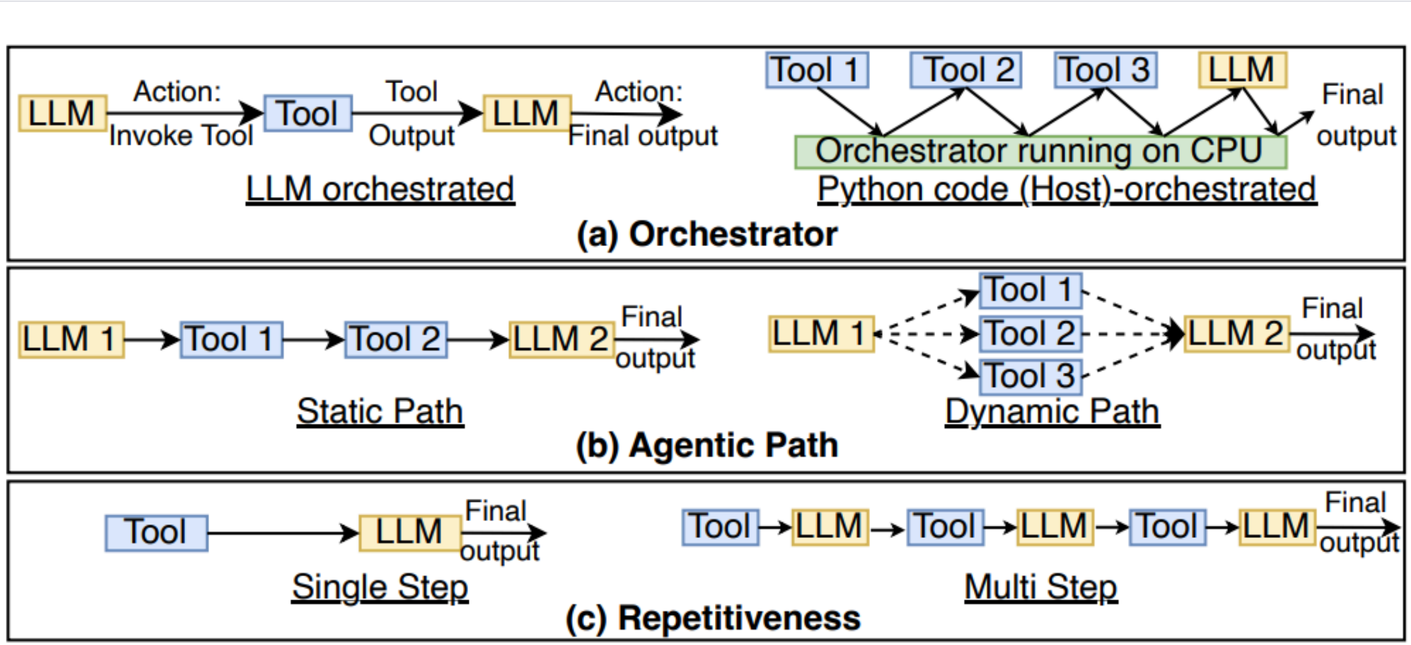


Figure 1. Compile-time Characterization of agentic AI on the basis of (a) Orchestrator (LLM/Host) (b) Agentic Path (Static/Dynamic) and (c) Repetitiveness (Single/Multi-step).

Orchestrator
`LLM-driven vs Host (CPU) Python`

Agentic path
`Static sequence vs Dynamic choice`

Repetitiveness
`Single-step vs Multi-step loop`

interleaves **GPU inference** with CPU tools — prompt+tools **repeat every step**.

Source: Raj, Kundu, Vohra, Wang, Krishna — \u201cTowards Understanding, Analyzing, and Optimizing Agentic AI Execution: A CPU-Centric Perspective\u201d (Georgia Tech · Intel).

© 2026 XCENA Inc.

Where Agentic Time Actually Goes

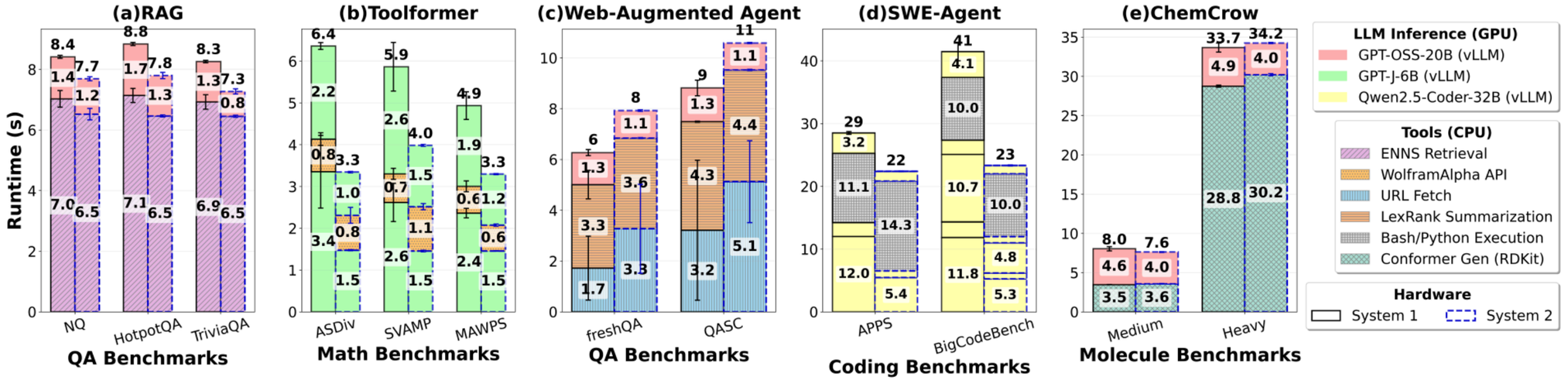


Figure 2. (a) End-to-end (E2E) latency for RAG (Haystack); (b) Toolformer; (c) Web-Augmented Agent (LangChain); (d) Mini-SWE-Agent; and (e) ChemCrow on the two different hardware systems (refer to Table 1).

CPU tools dominate
retrieval, URL fetch, RDKit conformer gen — often most of the wall-clock.

GPU = one slice
LLM inference is a minority of E2E in several apps — reuse cuts the prefill part.

Multi-step → recurs
the repeating prompt prefill returns every step — **prefix caching pays**.

Source: Raj, Kundu, Vohra, Wang, Krishna — \u201cTowards Understanding, Analyzing, and Optimizing Agentic AI Execution: A CPU-Centric Perspective\u201d (Georgia Tech · Intel).
Two bars/benchmark = two hardware systems; vendor-reported.

© 2026 XCENA Inc.

PART 2

Real-world Workload

Two production traces, up close — Mooncake and KVCache-in-the-Wild.

© 2026 XCENA Inc.

The Open-Source Request Trace

Listing 1: Request samples.

```
{
    "timestamp": 27482,
    "input_length": 6955,
    "output_length": 52,
    "hash_ids": [46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 2353, 2354]
}
{
    "timestamp": 30535,
    "input_length": 6472,
    "output_length": 26,
    "hash_ids": [46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 2366]
}
```

23,608
entries · 1 hour

512
tokens / hash block

4 fields, no text
`timestamp` · `input_length` · `output_length` · `hash_ids`

first **12 matching hash_ids** → shared prefix of 12×512 = **6,144 tokens**.

Source: Mooncake — first open trace for real-world KVCache reuse analysis (hashes include all preceding blocks).

© 2026 XCENA Inc.

Huge Input, Tiny Output

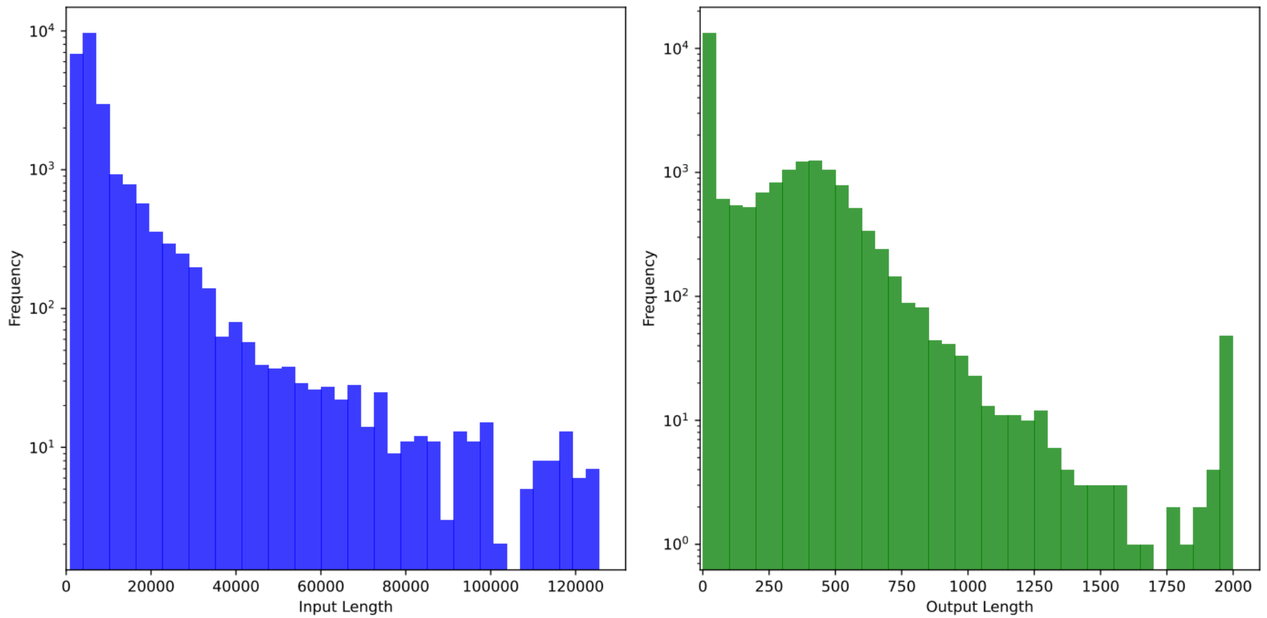


Figure 5: Input and output length distributions in the request trace.

7,590
avg input tok

182
avg output tok

~720 : 1
input-to-output ratio → extremely prefill-heavy.

tail past **120K** input (Kimi long-context) — exactly where prefix caching pays most.

Source: Mooncake, Figure 5 (representative pattern, not universal).

© 2026 XCENA Inc.

MOONCAKE · WHY 720:1 IS PREFILL-HEAVY

Input Scales Prefill, Output Scales Decode

ISL input sequence length — the prompt

OSL output sequence length — the generation

PREFILL Read the input

`work ∝ ISL`

One parallel pass over **every input token** — the cost grows with input length.

7,590 input tok · processed once

DECODE Generate the output

`work ∝ OSL`

One token per forward pass, repeated — the work grows with output length.

182 sequential steps

ISL : OSL ≈ 720 : 1 → ~98% of processed tokens are input → prefill dominates compute → the prefill KV is what's worth caching.

© 2026 XCENA Inc.

A Few Hot Blocks Carry the Reuse

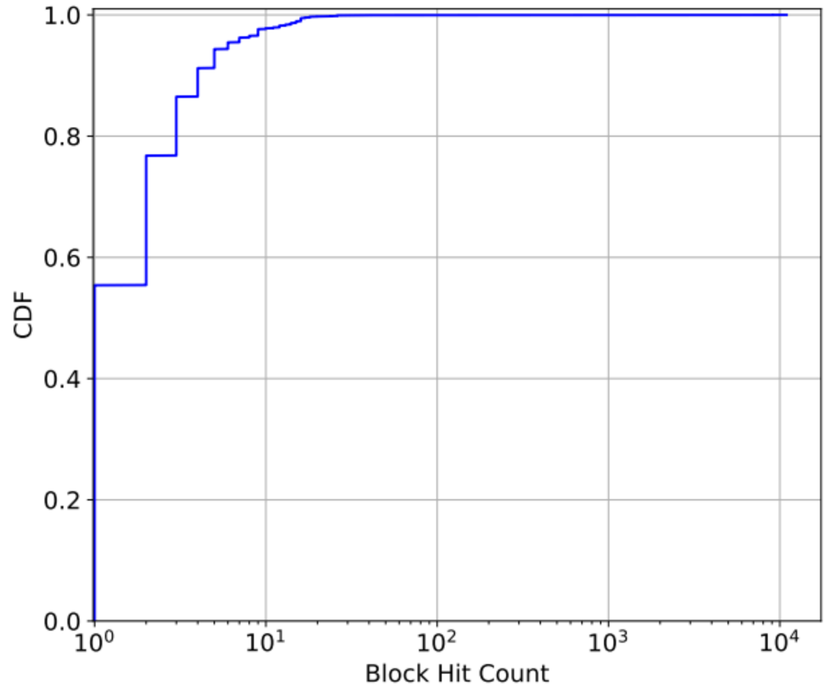


Figure 6: CDF (Cumulative Distribution Function) of the block hit count in the request trace.

>50%
blocks never reused

10^4
hits on hottest blocks

Replicate hot blocks
a small hot set dominates → replicate to avoid transfer congestion.

storage-side mirror of workload skew — build the system around the hot set.

Source: Mooncake, Figure 6.

© 2026 XCENA Inc.

LRU Wins; Capacity Plateaus at ~50K

Table 1: Cache hit rates under different cache policies and capacities.

Block capacity	Inf	100000	50000	30000	10000	1000
LRUCache	0.51	0.51	0.50	0.48	0.40	0.30
LFUCache	0.51	0.51	0.49	0.43	0.35	0.30
LengthAwareCache	0.51	0.50	0.48	0.42	0.35	0.30

Capacity
1K→50K blocks: **30%→50%** hit, then flat (sample subset).

LRU wins
temporal proximity → recency beats LFU & length-aware.

Simple recency + enough capacity captures **most of the benefit**.

Source: Mooncake, Table 1 (global pool; LRU / LFU / LengthAwareCache).

© 2026 XCENA Inc.

What a Collected Request Looks Like

```
// Figure 3 — anonymized request record
{ "timestamp": "202x-xx-yy 17:41:33.945",
"chat_id": "…abcdef…",
"parent_chat_id": "…778899",
"user_id": "100000…890",
"req_type": "text",
"num_input_tokens": 11026,
"num_output_tokens": 312,
"tokens_hashes_input": [1001,2002,…],
"tokens_hashes_output": [1234,2341,…] }
```

2 weeks, 2 traces

Feb 2025 + Dec 2024 from an Alibaba cluster — one representative day each, two typical workloads.

Privacy-preserving

no raw text/tokens; one model per trace. Hashes encode reuse, not content.

parent_chat_id → sessions · **tokens_hashes** → prefix reuse

Source: KVCache-in-the-Wild (Alibaba), Figure 3.

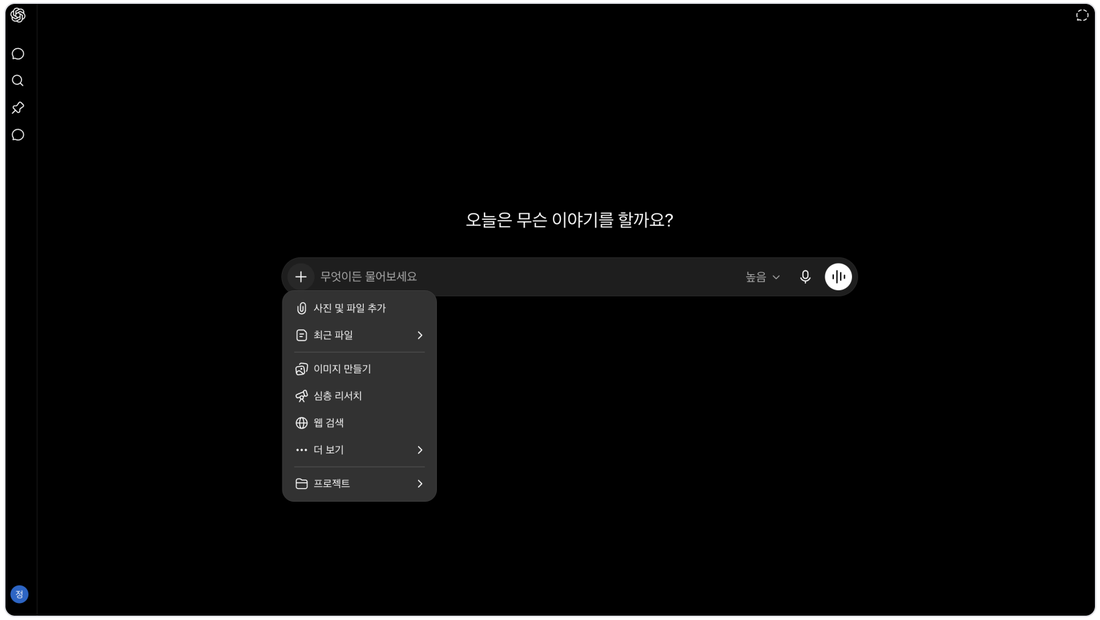
© 2026 XCENA Inc.

Trace A vs Trace B

TRACE A · to-C (consumer)

Human-in-the-loop

- cloud services: ChatBot (text), File, Multimodal, Search
- people interacting directly → human pace
- has request-type info



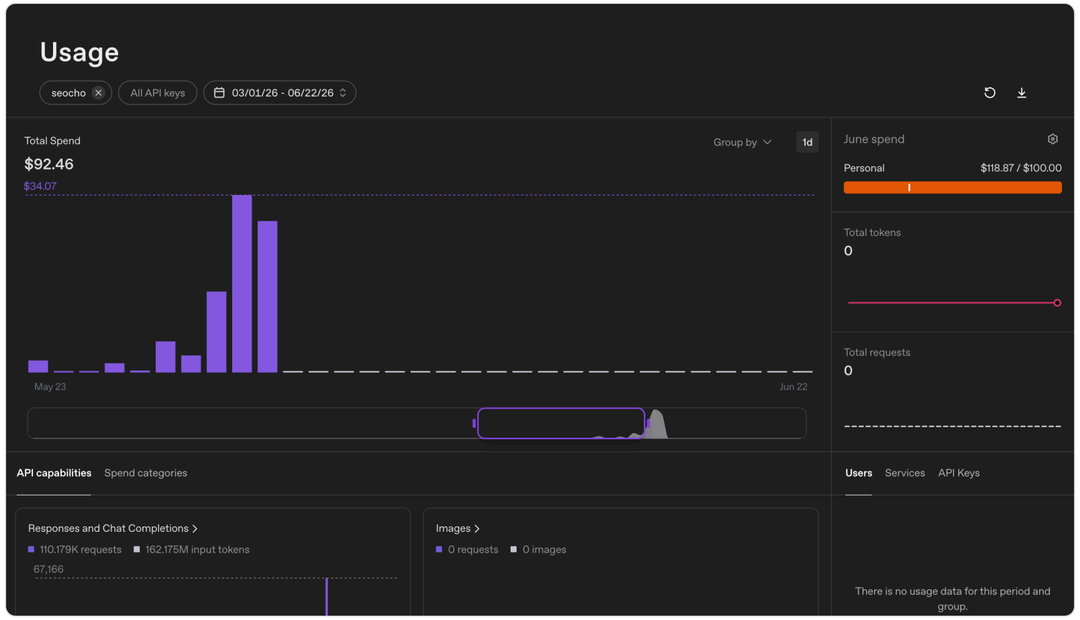
EXAMPLE · CONSUMER CHAT

reuse at conversation cadence

TRACE B · to-B (business)

Program-driven API

- OpenAI-compatible API calls from programs
- automation: translation, classification
- no request-type field (concatenated client-side)



EXAMPLE · API USAGE DASHBOARD

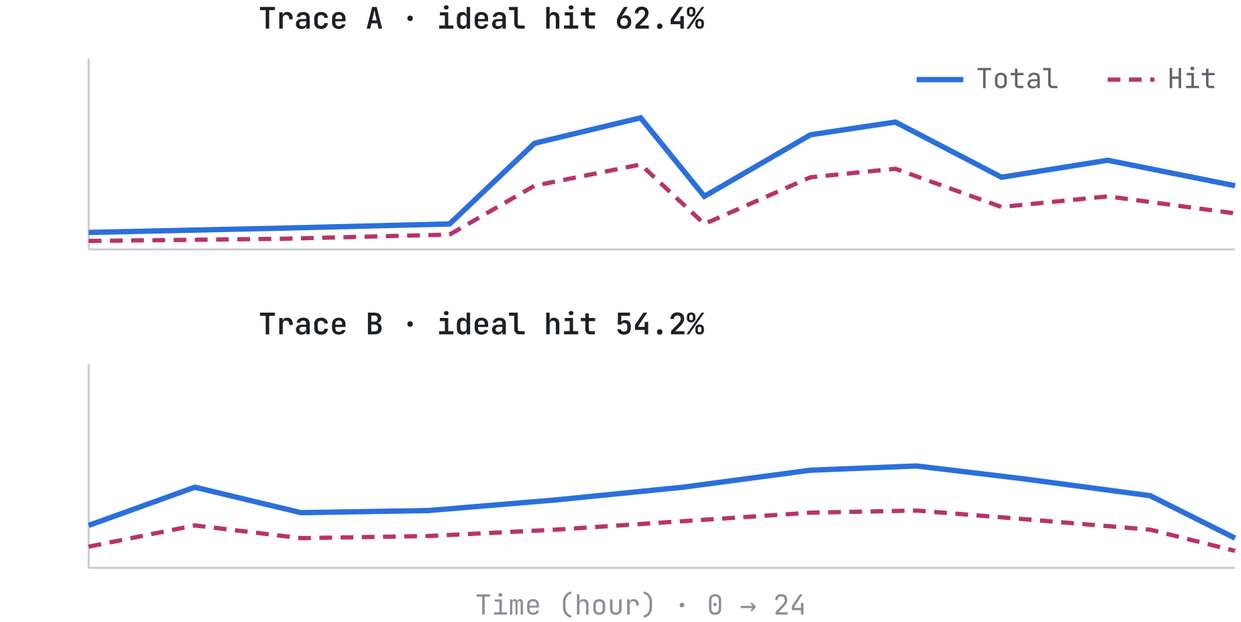
reuse from shared prompts, high QPS

Source: KVCache-in-the-Wild (Alibaba).

© 2026 XCENA Inc.

ALIBABA TRACE · FIGURE 4 · IDEAL HIT RATIO

How Much KV Can Be Reused?



62.4%
Trace A ceiling

54.2%
Trace B ceiling

Ideal = infinite cache. Still **below the 80%+** claimed on synthetic workloads — design for **54–62%**.

Source: KVCache-in-the-Wild, Figure 4 (hit = block reuses a prior cached KV).

© 2026 XCENA Inc.

ALIBABA TRACE · FIGURE 5 · WHO DRIVES HITS

Single-Turn Reuse Matters Too

TRACE A · to-C

Chatbot-dominated

78% requests are chatbot (text)

47–51% multi-turn ratio (most types)

TRACE B · to-B

Single-turn API

<0.1% multi-turn ratio

97% of hits from single-turn

Why single-turn still hits: programs hardcode identical **system prompts** (shared prefix) + high QPS (>10/s) → the same KV is reused constantly.

Source: KVCache-in-the-Wild, Figure 5.

© 2026 XCENA Inc.

Reuse Is Intra-User, Not Cross-User

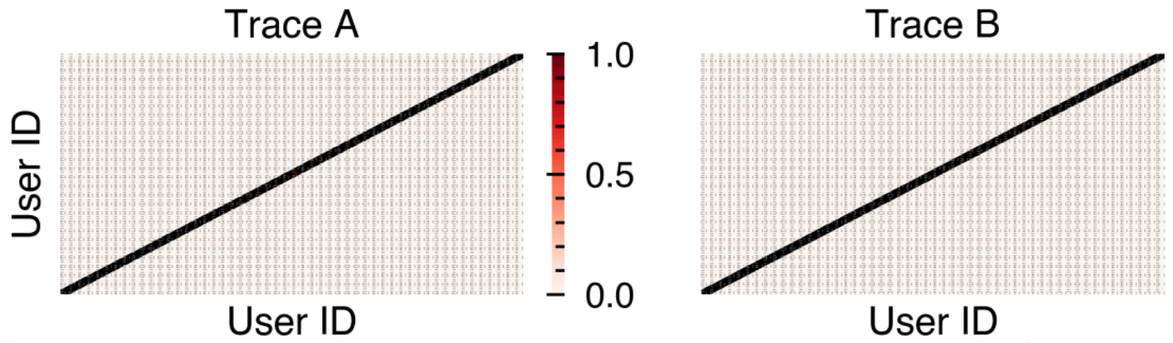


Figure 6: *An analysis of whether a user may hit the KV$ cached by another user. The darker the color, the higher the normalized hit count. Note that we have amplified the diagonal line in case it is too thin, since the trace provider serves numerous users.*

Diagonal = self-reuse
each user reuses their own KV — multi-turn or same system prompt.

Off-diagonal ≈ empty
users customize prompts → cross-user hits are rare.

Design for **intra-user reuse** — global LRU ignoring ownership is the wrong model.

Source: KVCache-in-the-Wild, Figure 6 (infinite-cache assumption · diagonal amplified).

© 2026 XCENA Inc.

A Few Users Drive Most Reuse

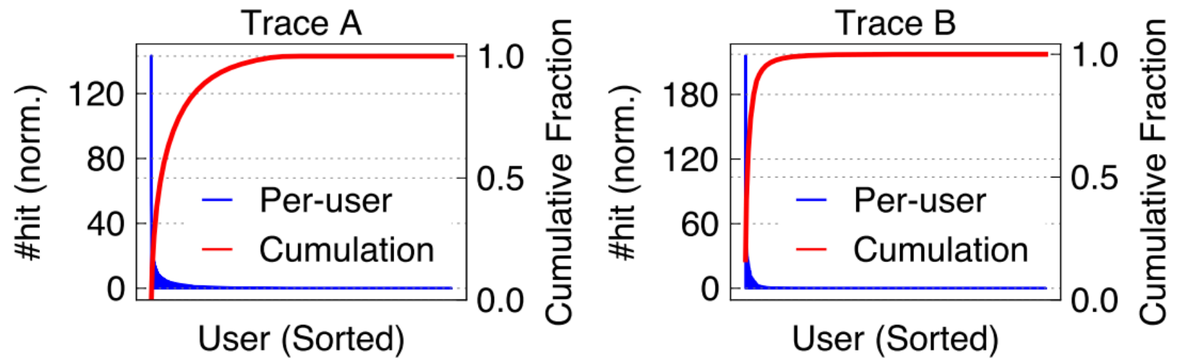


Figure 7: *An analysis of the normalized (norm.) hits contributed by different users in an ideal setup with infinite cache size.*

19%
Trace A reqs → 90% hits

4%
Trace B reqs → 90% hits

Why skewed

head users send most requests (B: 15% → 90%), and hits are intra-user; multi-turn users skew it further (A).

→ a modest cache holding the **head users hot KV** captures most of the benefit.

Source: KVCache-in-the-Wild, Figure 7 (ideal, infinite cache).

© 2026 XCENA Inc.

Multi-Turn Counts Vary Wildly

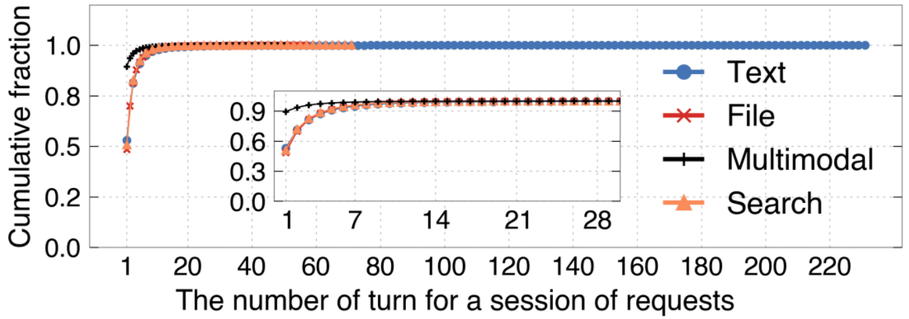


Figure 9: *Distribution of turn number of multi-turn requests on Trace A.*

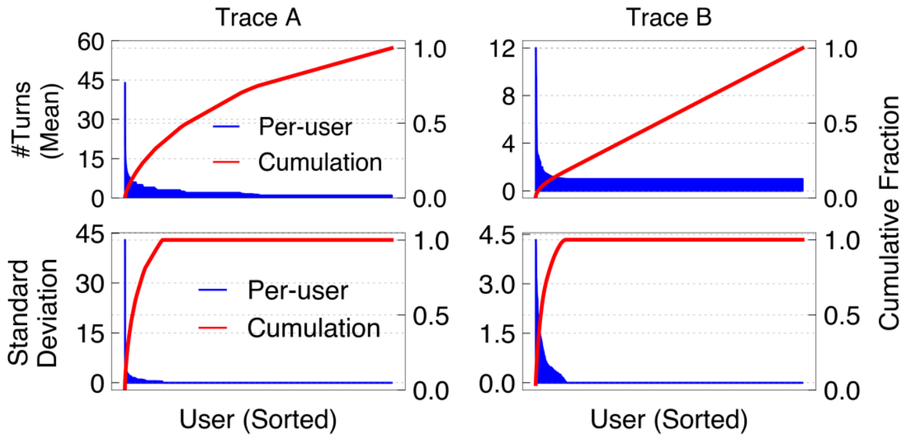


Figure 10: *An analysis of the mean (upper) and standard deviation (lower) of the number of turns of requests sent by each user. The users are sorted across to their mean number of turn (upper) and standard deviation (lower).*

Across sessions (Fig 9)

54% single-turn · P50=1, P90=5 · long tail to **232 turns** (232× median).

Across users (Fig 10)

top 10% have **8.9×** more turns than average; per-user deviation 1–44.

No single turn-count profile — cache must tolerate **both single-shot and 200-turn marathons**.

Source: KVCache-in-the-Wild, Figures 9 & 10.

© 2026 XCENA Inc.

Next-Turn Behavior Is Predictable

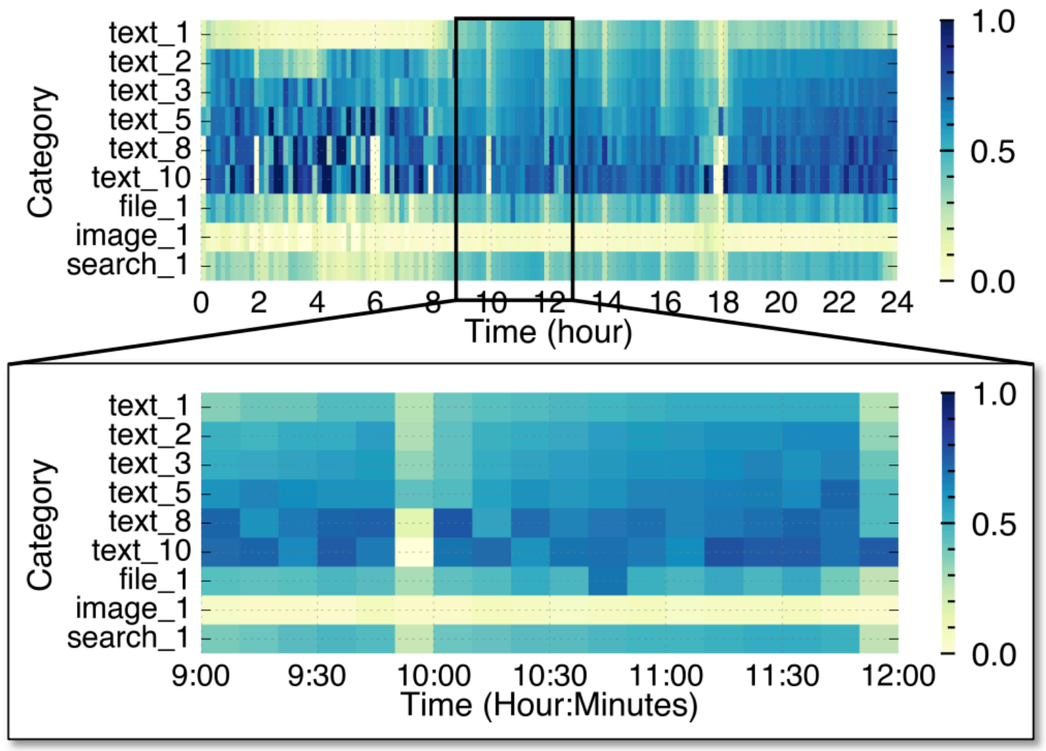


Figure 11: *An illustration of how the frequencies of issuing the next turn request change over time for different request types. The x-axis is the timeline. The workload categories are sorted according to their popularity.*

Stable over ~1 hour
horizontal bands: for a given category, next-turn rate holds steady within a time window.

Differs by category
vertical bands: type + turn-number = category, each with its own rate.

0.5 `single-turn mean` **0.8** `multi-turn mean`

predictable → forecast reuse, prefetch & size cache.

Source: KVCache-in-the-Wild, Figure 11 (10-min windows; 24h + 3h zoom).

© 2026 XCENA Inc.

KV Reuse Time Is Short

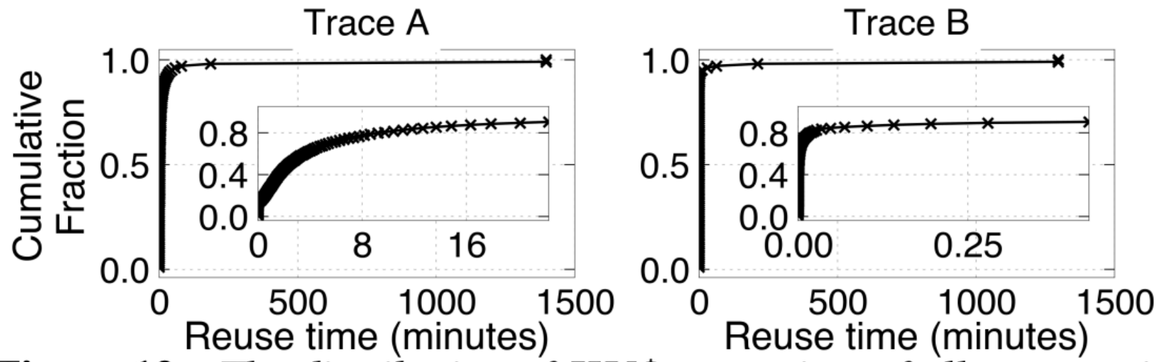


Figure 12: *The distribution of* KV$ *reuse time of all requests in Trace A and B.*

<10 min
Trace A · 80% of reuse

<10 s
Trace B · 80% of reuse

Human vs computer in the loop
people type next turns in minutes; programs fire next calls in seconds.

→ size cache to the **reuse window**, not max context — huge long-term storage is unnecessary.

Source: KVCache-in-the-Wild, Figure 12 (reuse-time CDF).

© 2026 XCENA Inc.

Locality Differs by Workload Type

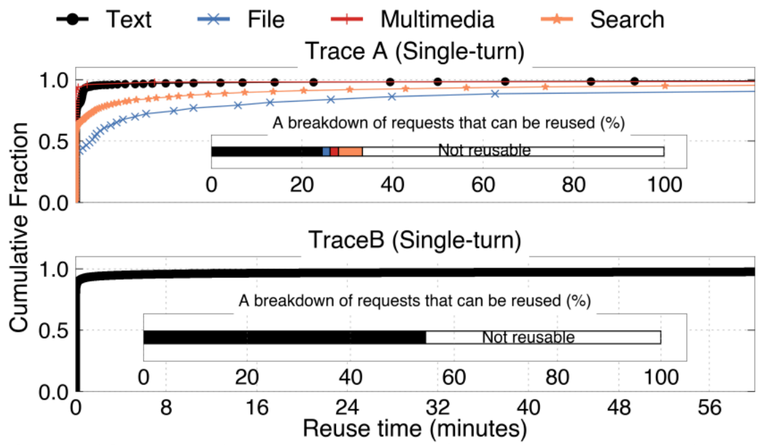


Figure 13: *The distribution of* KV$ *reuse time of single-turn requests in Trace A and B. B has one category of request (API).*

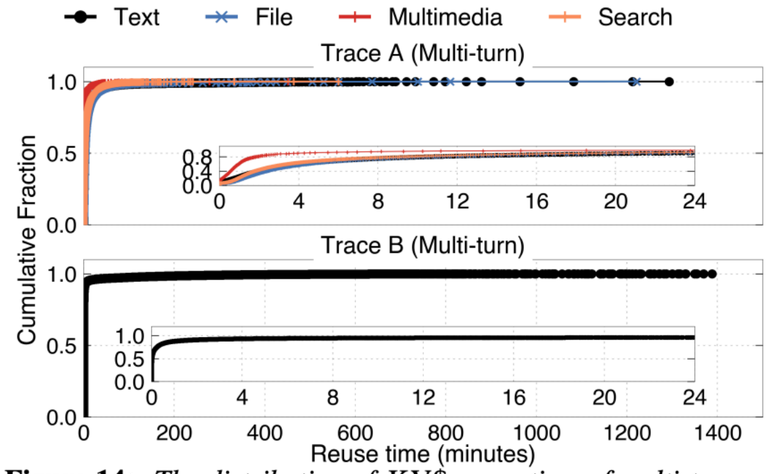


Figure 14: *The distribution of* KV$ *reuse time of multi-turn requests in Trace A and B. B has one category of request (API).*

Single-turn flips (Fig 13)
to-C reusable **<30%** · to-B **>50%** — opposite locality.

Reuse time by type (Fig 14)
File > Text > Multimodal · Search second-longest.

Locality depends on **turn structure + modality** — one cache policy leaves value on the table.

Source: KVCache-in-the-Wild, Figures 13 & 14.

© 2026 XCENA Inc.

Cache From the Beginning

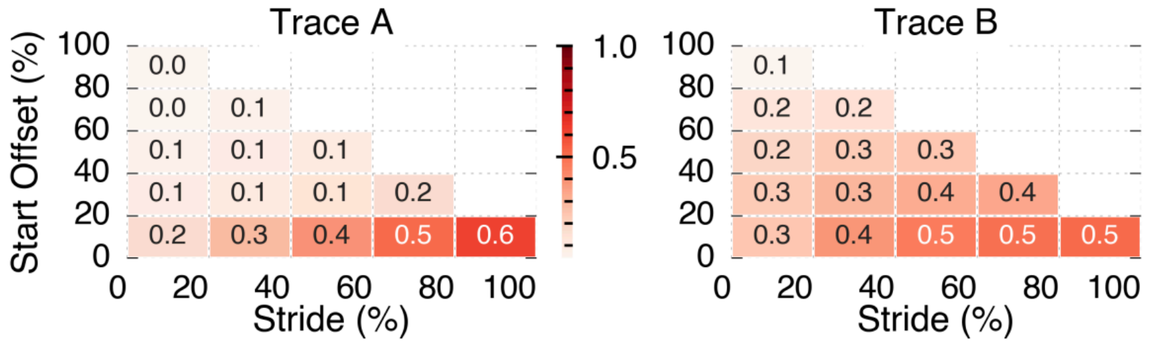


Figure 16: *Overall spatial locality of two traces.*

Offset 0 wins
hit peaks at the bottom row (start = token 0): up to **0.6** A / **0.5** B; fades as offset rises.

Only prefixes share
identical prefixes share KV → middle/tail caching yields little.

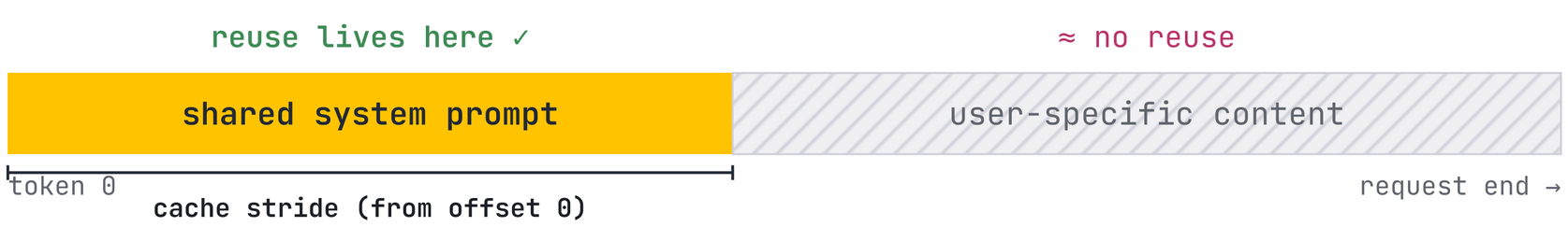
Tail-caching to overlap load **sacrifices spatial locality** — cache prefixes, from token 0.

Source: KVCache-in-the-Wild, Figure 16 (stride × start-offset hit-ratio heatmap).

© 2026 XCENA Inc.

Spatial Locality Is Workload-Aware

reading the axes — one request, split by token position



x-axis · cache stride
how big a slice you cache

y-axis · start offset
which token you begin at

reuse only on the **shared prefix** → caching from token 0 wins.

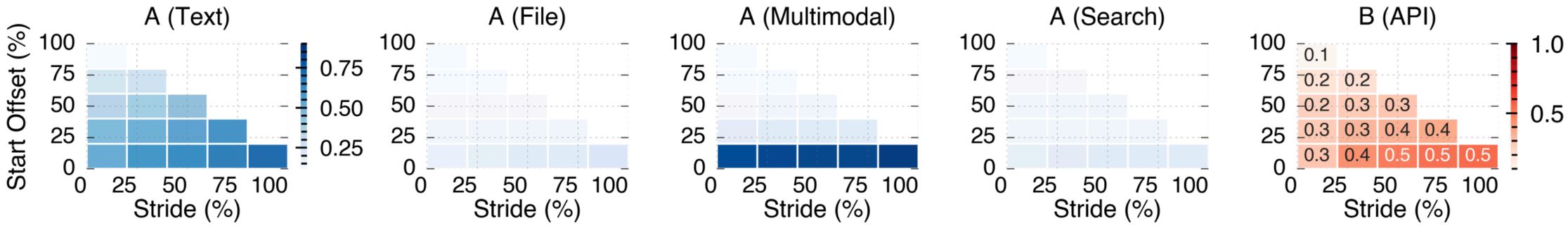


Figure 18: *The spatial locality of single-turn requests.*

Text & Multimodal
clear locality — fixed system prompt prefix.

File & Search
almost none — prompt depends on user input.

Trace B plateaus
peak at **50% stride** → sys prompt < half the request.

Source: KVCache-in-the-Wild, Figure 18. Section takeaway: reuse is real but workload-specific.

© 2026 XCENA Inc.

KV Blocks Die Fast

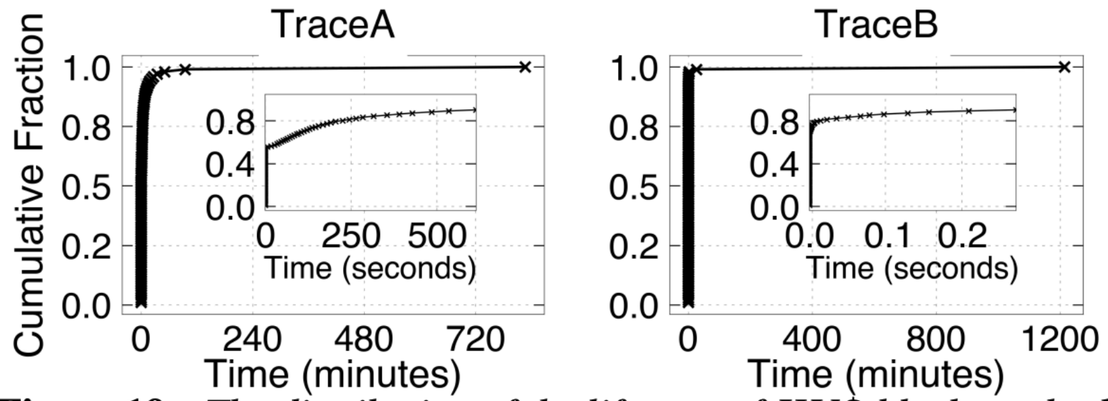


Figure 19: *The distribution of the lifespan of* KV$ *blocks on both traces.*

612 s
Trace A · 90% dead by

0.3 s
Trace B · 90% dead by

Why so short
short reuse times + few turns → when a session ends, its KV dies.

B shorter overall (system-prompt hits), but rare outliers to **1,200 min** vs 800 in A.

Source: KVCache-in-the-Wild, Figure 19 (block-lifespan CDF).

© 2026 XCENA Inc.

Lifespan Is Predictable Over Time

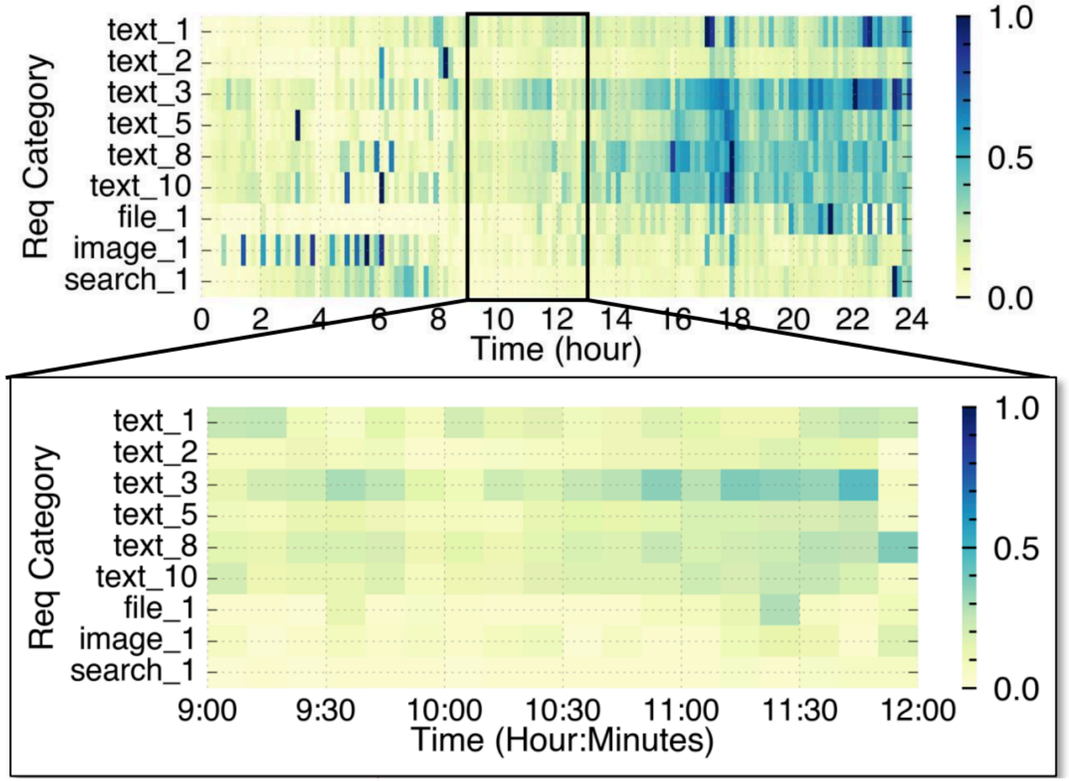


Figure 20: *An illustration of how the mean lifespan of* KV$ *blocks changes over time for different request categories. Note that the x-axis represents the timeline. The request categories are sorted according to their usage.*

Stable short-term
drifts over 24h, but steady when zoomed to 9–11am → predictable per category.

Predict → save cost
forecast lifespan from history → right-size cache; DRAM is expensive.

Workloads are **reusable, short-lived, skewed, predictable** — the structure LMCache exploits.

Source: KVCache-in-the-Wild, Figure 20 (mean lifespan heatmap; 24h + 3h zoom).

© 2026 XCENA Inc.

Per-Request KV Is Small vs HBM

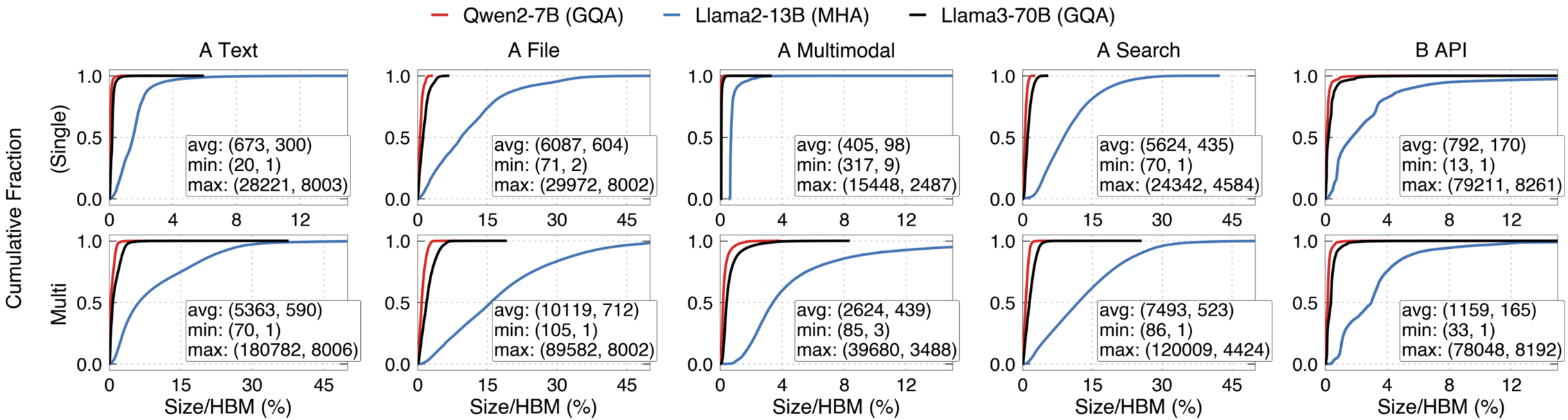


Figure 21: *The distribution of* KV$ *size of requests on different traces for Single and Multi-turn requests. The size is normalized to the GPU memory (HBM) available for serving. The number in the tuple inside each graph represents the length of input and output.*

GQA = tiny
Qwen2-7B Text: **0.09%** single / **0.56%** multi of HBM.

MHA = biggest
Llama2-13B (blue) far right; GQA/MLA store less per token.

~1 min to fill HBM
at a few req/s → within reuse distance → offload becomes reuse.

Source: KVCache-in-the-Wild, Figure 21 (KV size / HBM; tuple = input,output length).

© 2026 XCENA Inc.

GQA Needs Little Cache; MHA Needs A Lot

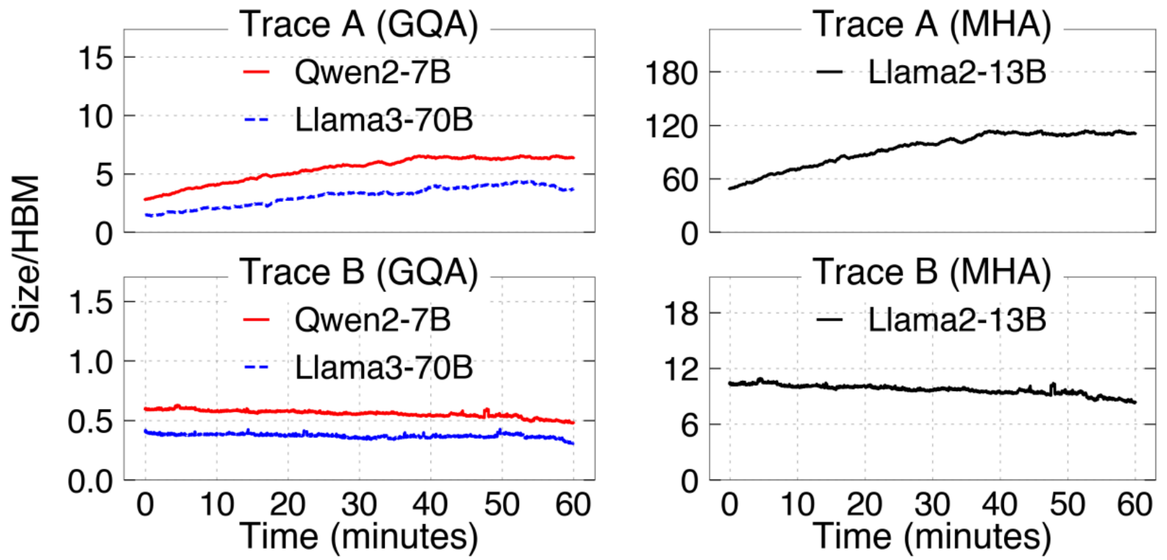


Figure 22: *An analysis of the KV$ cache size required to achieve an ideal cache hit ratio on different models and traces. The data is collected from 10:00 a.m. to 11:00 a.m.*

GQA: ~4× HBM (A)
Llama3-70B needs ~4× — covered by 1 TB CPU RAM / 8 A100 (~128 GB/GPU). No SSD needed.

Trace B: < 1× HBM
smaller than reserved HBM → GPU caching may suffice.

MHA: up to ~120×
Llama2-13B's big per-token KV → eviction policy still matters. enter LMCache.

Source: KVCache-in-the-Wild, Figure 22 (cache size / HBM, 10-11am).

© 2026 XCENA Inc.

PART 3

vLLM & LMCache

How vLLM serves and manages KV — and the connector interfaces LMCache integrates with.

© 2026 XCENA Inc.

The Problem — and Three Fixes We'll Cover

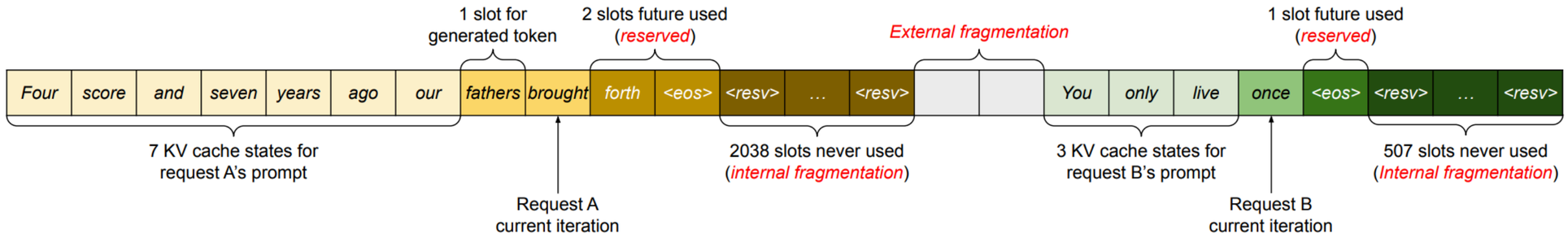


Figure 3. KV cache memory management in existing systems. Three types of memory wastes – reserved, internal fragmentation, and external fragmentation – exist that prevent other requests from fitting into the memory. The token in each memory slot represents its KV cache. Note the same tokens can have different KV cache when at different positions.

Reserved, internal & external fragmentation waste KV memory and block other requests. Many techniques tackle this — we'll look briefly at three:

① SCHEDULING

Continuous batching

admit + retire every step; prefill & decode share one batch.

② PREFILL

Chunked prefill

split long prefill into chunks, interleave with decode → flat ITL.

③ MEMORY

PagedAttention

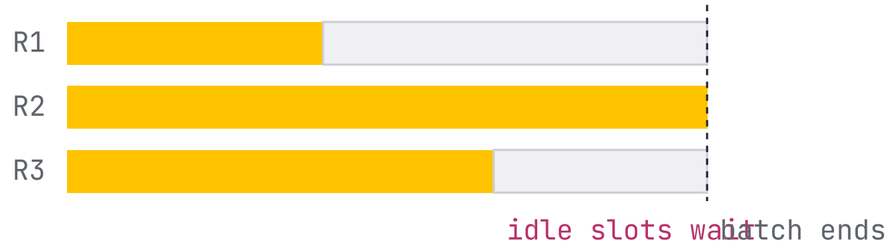
KV in fixed-size blocks, not a contiguous slab → no fragmentation.

Fig. 3 — KV memory waste in existing systems (Kwon et al., vLLM).

© 2026 XCENA Inc.

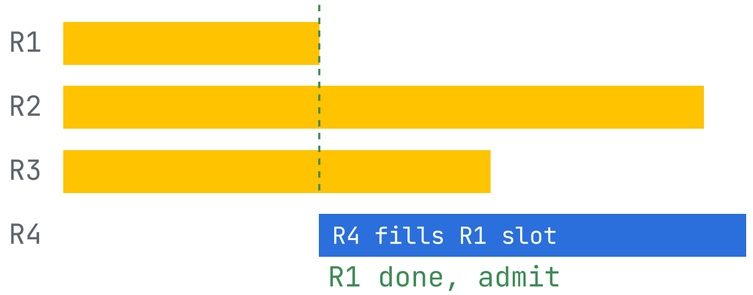
Re-form the Batch Every Step

AS-IS Static batching



- Batch is **fixed at launch** — all requests start together.
- GPU runs until the **slowest** request in the batch finishes.
- Finished slots sit **idle** until the batch ends.
- New requests **wait in queue** for the whole batch.

TO-BE Continuous batching



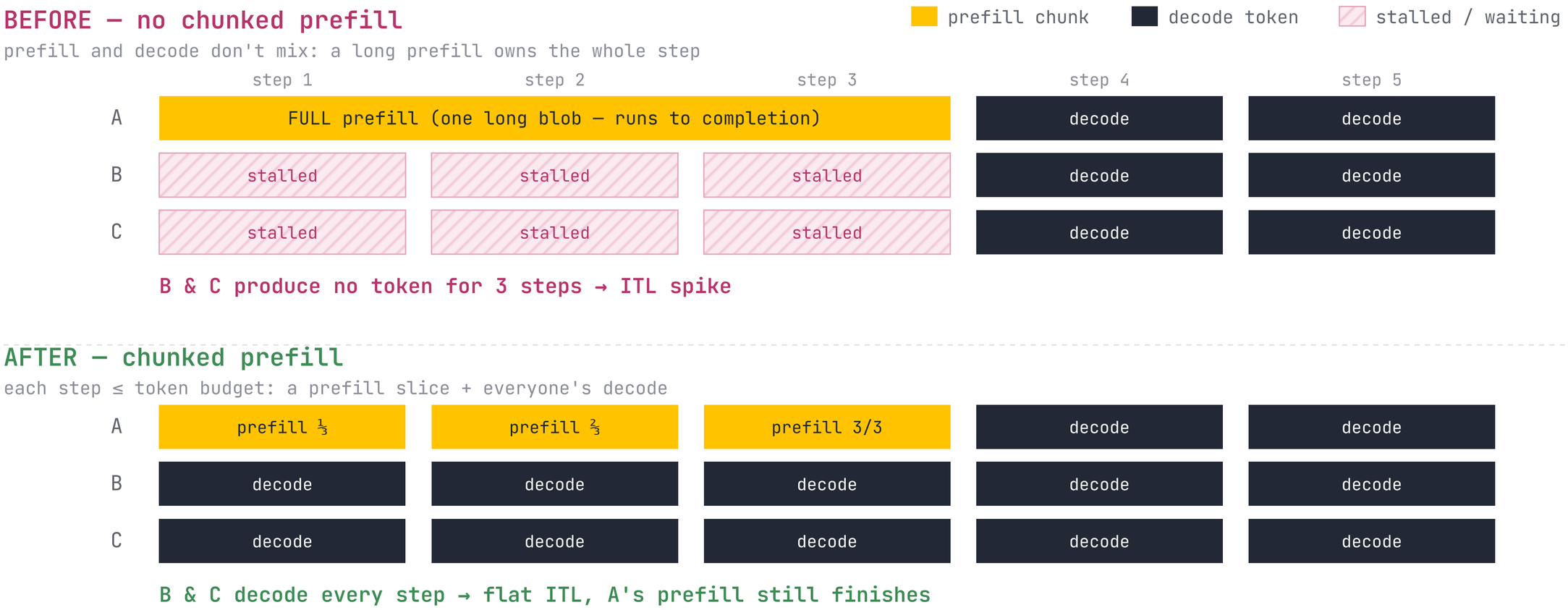
- Scheduler **re-forms the batch every decode step** (iteration-level).
- A finished request **leaves immediately**, freeing its slot.
- A waiting request is **admitted into the freed slot** that step.
- New prefill + ongoing decode **mix in one batch** → GPU stays full.

Iteration-level scheduling → **no bubble between requests**, GPU stays saturated.

© 2026 XCENA Inc.

VLLM · CHUNKED PREFILL

Slice the Prefill, Keep Decodes Flowing



© 2026 XCENA Inc.

Block Table: Logical → Physical KV Blocks

Request A

Prompt: *"Four score and seven years ago our"*

Outputs: *"fathers"* → *"brought"* → …

① prompt ② step 2 ③ step 3

Logical KV blocks

Block 0	① *Four*	① *score*	① *and*	① *seven*
Block 1	① *years*	① *ago*	① *our*	② *fathers*
Block 2	③ *brought*			
Block 3				

Block Table

physical block #	# filled
⑦ 7	① 4
① 1	①3 → ④②
③ 3	③ 1
–	–

Physical KV blocks

(scattered in GPU DRAM)

Block 0				
Block 1	① *years*	① *ago*	① *our*	② *fathers*
Block 2				
Block 3	③ *brought*			
Block 4				
Block 5				
Block 6				
Block 7	① *Four*	① *score*	① *and*	① *seven*
Block 8				

Logical = contiguous
filled left→right; last block reserves room

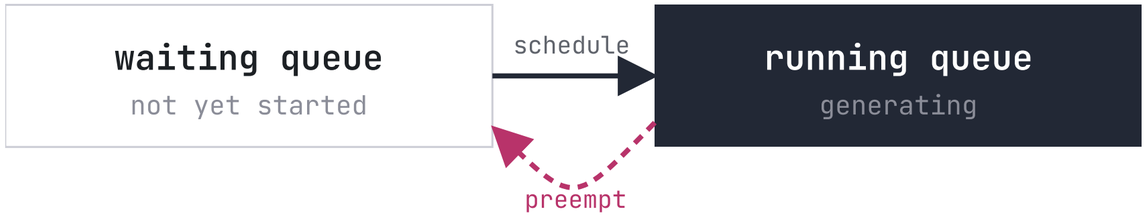
Table maps each block
logical → physical # + # filled

Grow on demand
no up-front reserve → ~no waste

After Fig. 6 — Block table translation in vLLM (Kwon et al., PagedAttention / vLLM).

© 2026 XCENA Inc.

Two Queues, and What Happens Under Pressure



FCFS · all-or-nothing · pool full → evict a victim

Preempt → recompute
drop victim's KV; recompute on resume. Cheap evict, costs compute.

Preempt → swap
copy KV to CPU and back. Bandwidth, not compute; swap space ≤ GPU KV pool.

Either way KV is lost or moved → **LMCache gives it a bigger, reusable home.**

© 2026 XCENA Inc.

Block Allocator, Ref-Counts, Prefix Cache

Block allocator
CPU + GPU block allocators; block-table entry = physical block + # filled slots.

Reference counting
shared blocks freed only at refcount 0 → safe copy-on-write.

Automatic prefix caching
hash blocks; matching prefix → reuse resident blocks, no recompute.

THE LIMIT

It only sees **one GPU's HBM**.

The prefix cache is capped by the KV pool, and dies on eviction or restart — no cross-instance, no persistence.

© 2026 XCENA Inc.

Paged Memory → Many Tiny I/Os

paged KV = many tiny, non-contiguous pages

4 GB/s

bandwidth st

62.5 KB each (Llama-3.1-8B) · scattered

---

coalesce → one large, contiguous transfer

≥ 16 MB batched KV

49 GB/s

link saturat

old path: torch.save / copies → sub-1 GB/s, no zero-copy

Table 1 · transfer size vs throughput (RCCL)

MESSAGE SIZE	THROUGHPUT	↑ with size
64 KB	4 GB/s	
256 KB	13 GB/s	
1 MB	30 GB/s	
10 MB	46 GB/s	
16 MB knee	49 GB/s	
100 MB	49 GB/s	

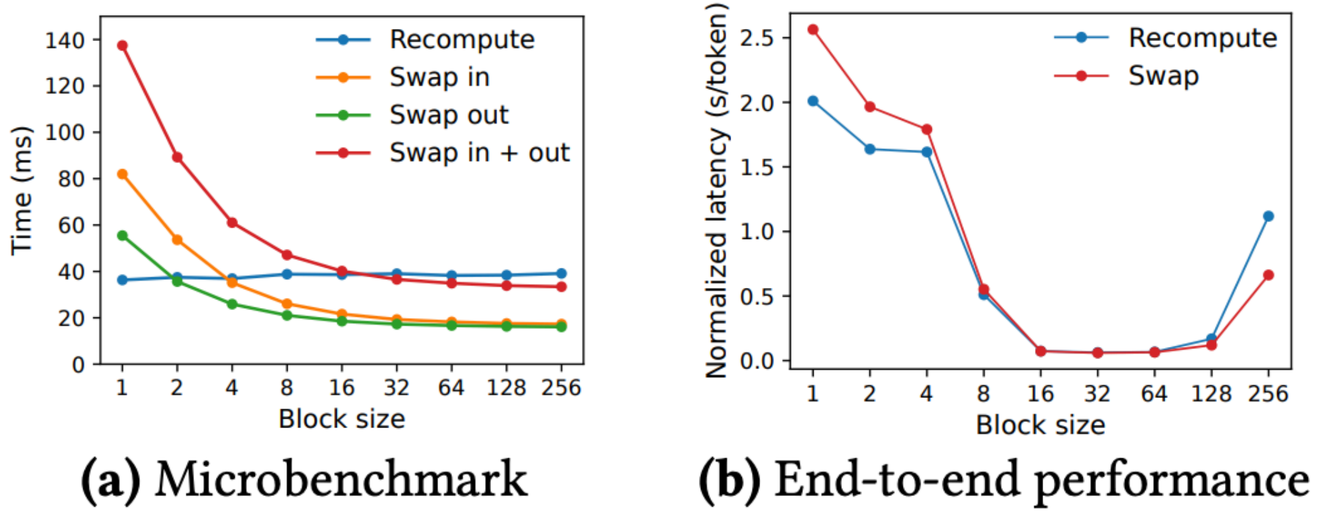
≥16 MB saturates a 400 Gb NIC · 1–2 MB ≈ 75–80% of PCIe 5.0.

LMCache must **coalesce scattered pages into large, contiguous transfers** — or the storage tier is bandwidth-starved.

Source: LMCache — Challenge #1 (I/O inefficiency under paged memory); Table 1 after UCCL Team (2025).

© 2026 XCENA Inc.

Recompute vs Swap — Block Size Decides



(a) Microbenchmark (b) End-to-end performance

Figure 19. (a) Overhead of recomputation and swapping for different block sizes. (b) Performance when serving OPT-13B with the ShareGPT traces at the same request rate.

Small blocks → recompute
swap = many tiny CPU↔GPU copies, can't fill PCIe.

Large blocks → swap
recompute cost is flat (ignores KV blocks); 16–64 comparable.

Bounded gap
recompute never > ~20% of swap's latency.

A sweet spot in **GPU + CPU** memory — but growing context needs more. **LMCache extends recovery to a storage tier.**

Fig. 19 — recompute vs swap, OPT-13B on ShareGPT (Kwon et al., vLLM §7.3).

© 2026 XCENA Inc.

What LMCache Fixes

vLLM ALONE

- cache lives in **one GPU's HBM**
- bounded by the small KV pool
- dies on eviction / restart
- not shared across instances

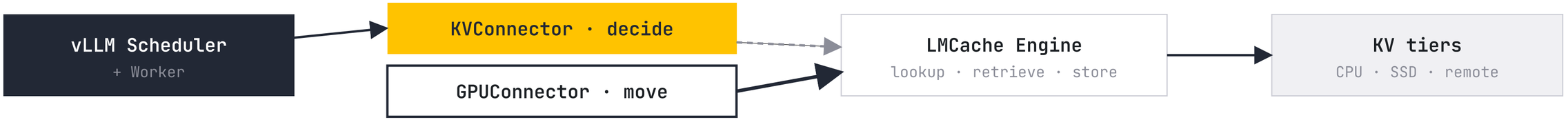
WITH LMCACHE

- KV in **CPU · disk · remote · CXL**
- far larger effective cache
- survives restart (persistent tiers)
- shared across engines

Same reuse idea — **lifted off the single GPU**, via vLLM's connector interface, no fork.

© 2026 XCENA Inc.

Two Interfaces: Decide, then Move



flow on a hit:

① KVConnector reports hit + reserves blocks → ② load before forward →

③ GPUConnector scatters cached KV into GPU slots → ④ skip prefill, generate

—— data (KV) ---- control

© 2026 XCENA Inc.

Three Scheduler Methods: When & Why

ONE SCHEDULER STEP

- **New request → waiting queue**
- ① **get_num_new_matched_tokens**
  LMCache: H tokens cached → **skip H prefill**
- **Allocate GPU blocks · waiting → running**
- ② **update_state_after_alloc**
  record which blocks hold the hit
- ③ **build_connector_meta**
  emit RETRIEVE / STORE ops
- ▸ **SchedulerOutput → Worker**

① `get_num_new_matched_tokens(request, num_computed_tokens)`

Why: reports how many tokens LMCache can supply — the number that turns a cache hit into **skipped prefill**.

```
in: token_ids, num already computed
out: (num_matched | None, is_async)
```

② `update_state_after_alloc(request, blocks, num_external_tokens)`

Why: blocks were just reserved — this records **which physical GPU blocks** the hit loads into, so the worker knows the destination.

```
in: request, allocated block_ids
out: — (state stored on connector)
```

③ `build_connector_meta(scheduler_output)`

Why: serializes this step plan into metadata the worker executes — the hand-off from **decide → move**.

```
in: scheduler_output
out: KVConnectorMetadata
  {req → RETRIEVE/STORE ops,
block_ids}
```

Scheduler side **decides** what to move — runs every step, moves nothing itself.

© 2026 XCENA Inc.

How the KV Actually Lands in the GPU

`start_load_kv(ctx) · wait_load_kv(…, layer)`

Load hit KV into GPU blocks before forward; wait per layer.

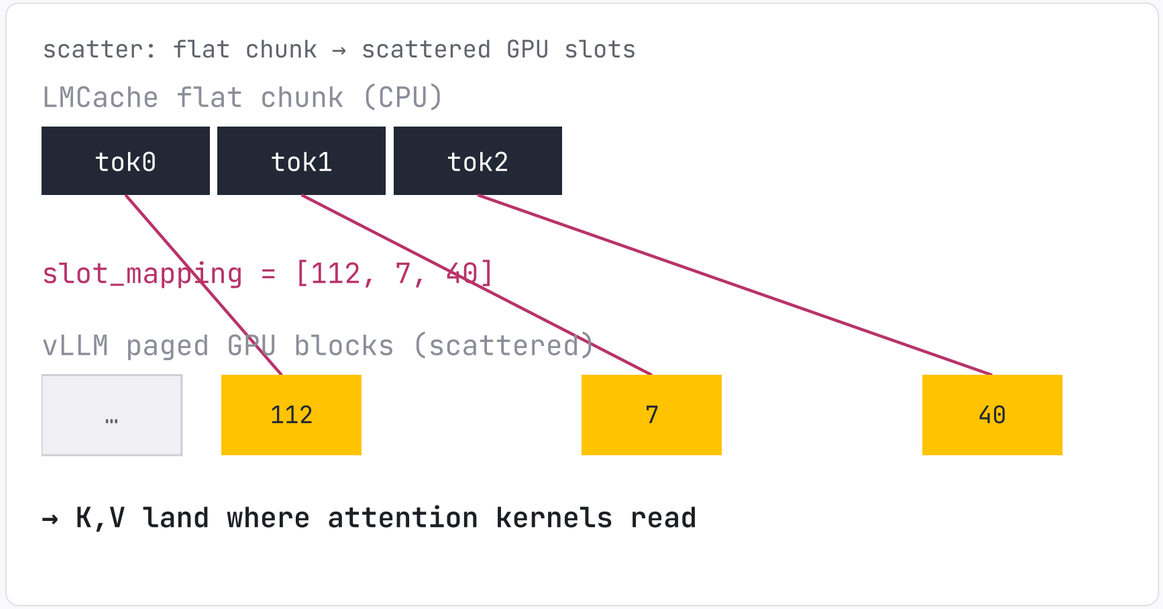
```
in: kv_caches, block_ids,
slot_mapping
out: KV written into kv_caches
```

`start_store_kv(…) · wait_store_kv(…, layer)`

Gather new KV from GPU slots → hand to LMCache to save.

```
in: kv_caches, slot_mapping
out: chunks stored
```

```
slot_mapping[t] = block_id · block_size + offset
one physical KV slot per token · applied every layer
```



© 2026 XCENA Inc.

Non-Contiguous GPU ↔ Contiguous Pool

GPU · per-layer, non-contiguous

each layer at its own base address → block tokens scattered

LMCache pool / storage · contiguous

one chunk, packed end-to-end

Gather Write

store: scattered GPU → contiguous pool

Scatter Read

retrieve: contiguous pool → scattered GPU

slot_mapping[i] = which slot each token goes to

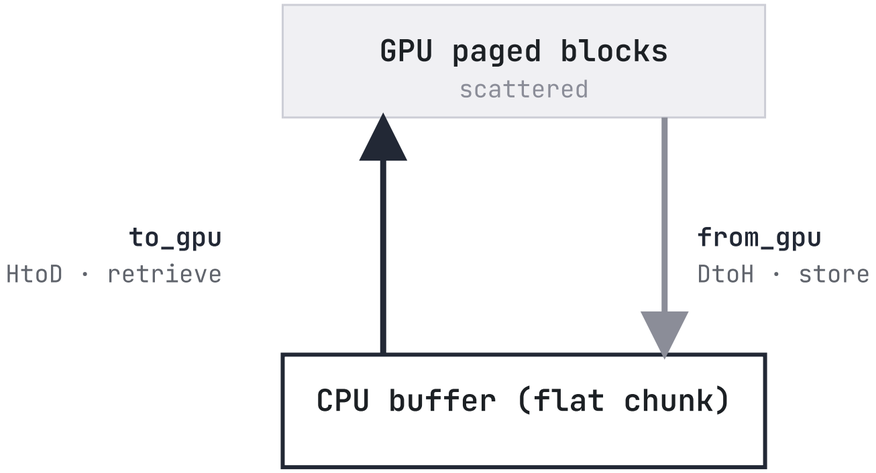
applied identically across every layer

GPUCONNECTOR · SCATTER / GATHER

A single 16-token block in Qwen-32B (GQA) = **128 non-contiguous 20 KB transfers** → why dedicated CUDA kernels matter.

© 2026 XCENA Inc.

GPUConnector — Scatter into the Right Slots



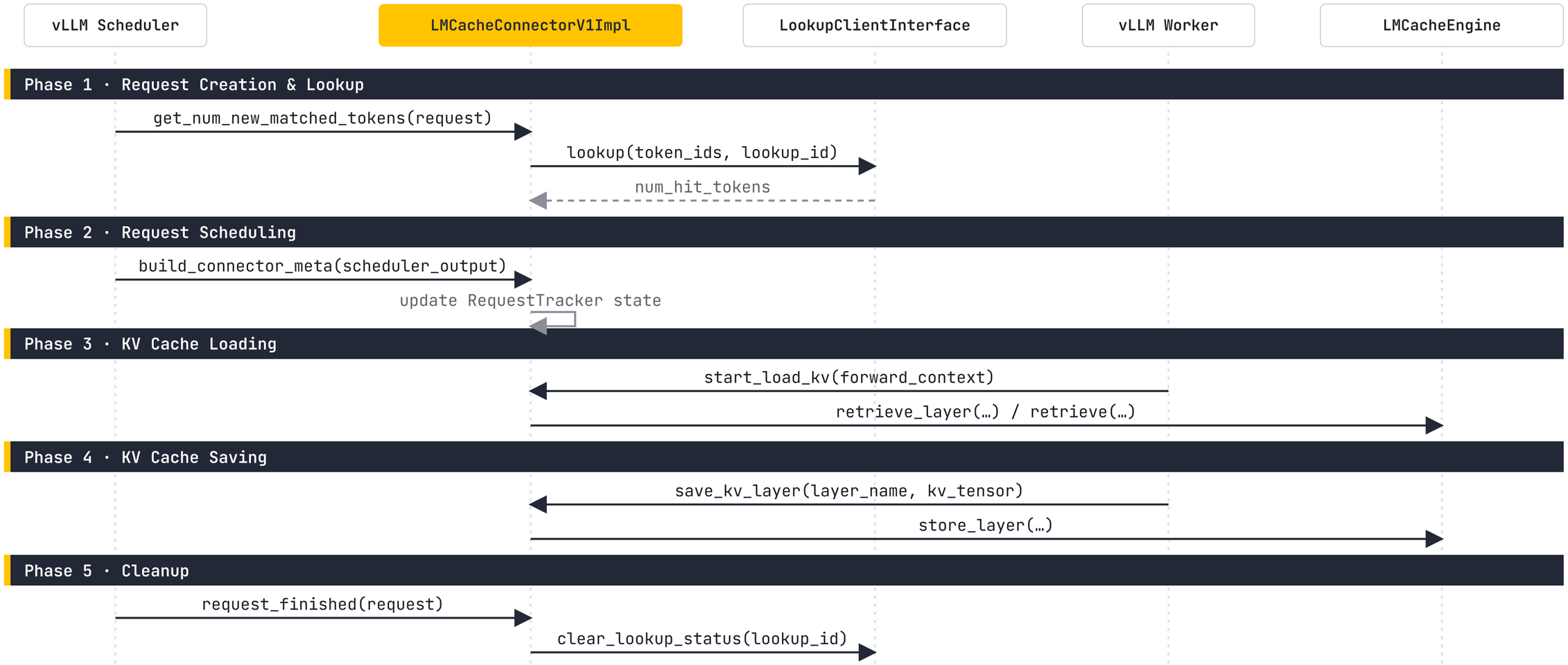
batched_to_gpu()
retrieve: CPU → paged GPU blocks (HtoD).

batched_from_gpu()
store: GPU → CPU buffer (DtoH).

```
slot = block_id·bs + offset → scatter every layer
```

© 2026 XCENA Inc.

One Request, Five Phases



Scheduler drives phases 1 · 2 · 5 · worker drives 3 · 4 · **solid = call**, dashed = return. The connector is the hub throughout.

© 2026 XCENA Inc.

PART 4

LMCache at a Glance

What LMCache is, where it sits in vLLM, and the two ways it deploys.

© 2026 XCENA Inc.

End-to-End System Workflow

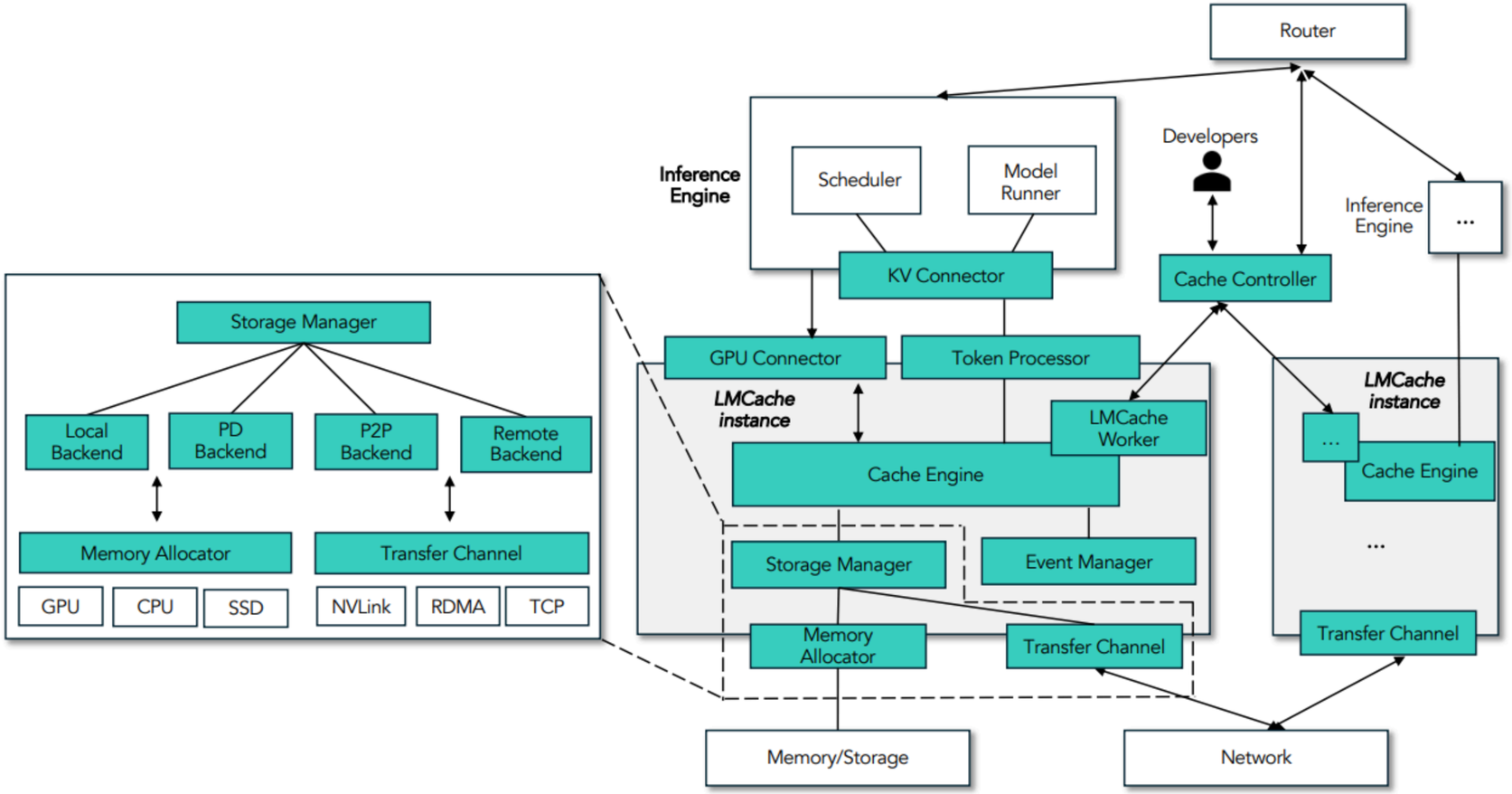


Figure 6: *End-to-end system workflow for* LMCACHE.

Connectors → Cache Engine → Storage Manager → backends (local · PD · P2P · remote) over NVLink / RDMA / TCP. Source: LMCache paper, Figure 6. We unpack each box next.

© 2026 XCENA Inc.

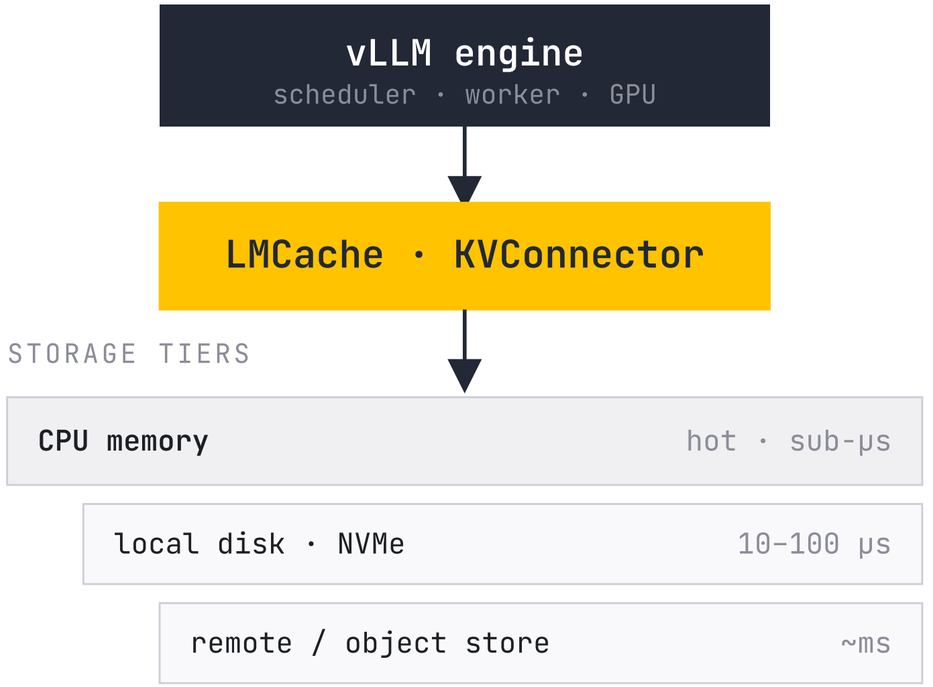
THE LAYER

LMCache: A KV Cache Tiering Layer

LMCache plugs into vLLM through the **KVConnector** interface and extends the cache **beyond the GPU**.

A hot tier in CPU memory, cold tiers below: disk, remote stores, object storage. Any prefix computed anywhere can be loaded back.

The question for the rest of this talk: **how do the bytes move?**



© 2026 XCENA Inc.

MAP OF THE TALK

Two Deployment Modes

IN-PROCESS MODE

In-Process

In-process with vLLM — the paper's core design. Storage backends own their buffers, synchronous transfers, serde in the hot path.

1 process · sync · serde inline

MULTI-PROCESS MODE

Multi-Process

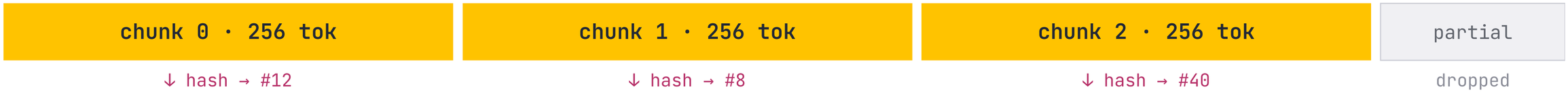
Separate LMCache process shared by many workers. Central memory manager, asynchronous L1↔L2 storage, raw tensors.

2 processes · async · shared L1

© 2026 XCENA Inc.

The Chunk — LMCache's Building Block

token sequence → grouped into 256-token chunks → each hashed to a key



Coarser than a block

vLLM block = 16 tok (GPU). LMCache chunk = **256 tok** — the cache's unit of store / reuse / transfer.

Hashed → a key

same text → same key → reusable on **any engine**, anytime — not just same-GPU prefix.

Big transfers

move a whole chunk per CUDA copy → fills the bus (the **400 vs 88 Gbps** gap).

© 2026 XCENA Inc.

TOKEN PROCESSOR · CHUNKEDTOKENDATABASE

Tokens → Keys, via a Prefix-Hash Chain

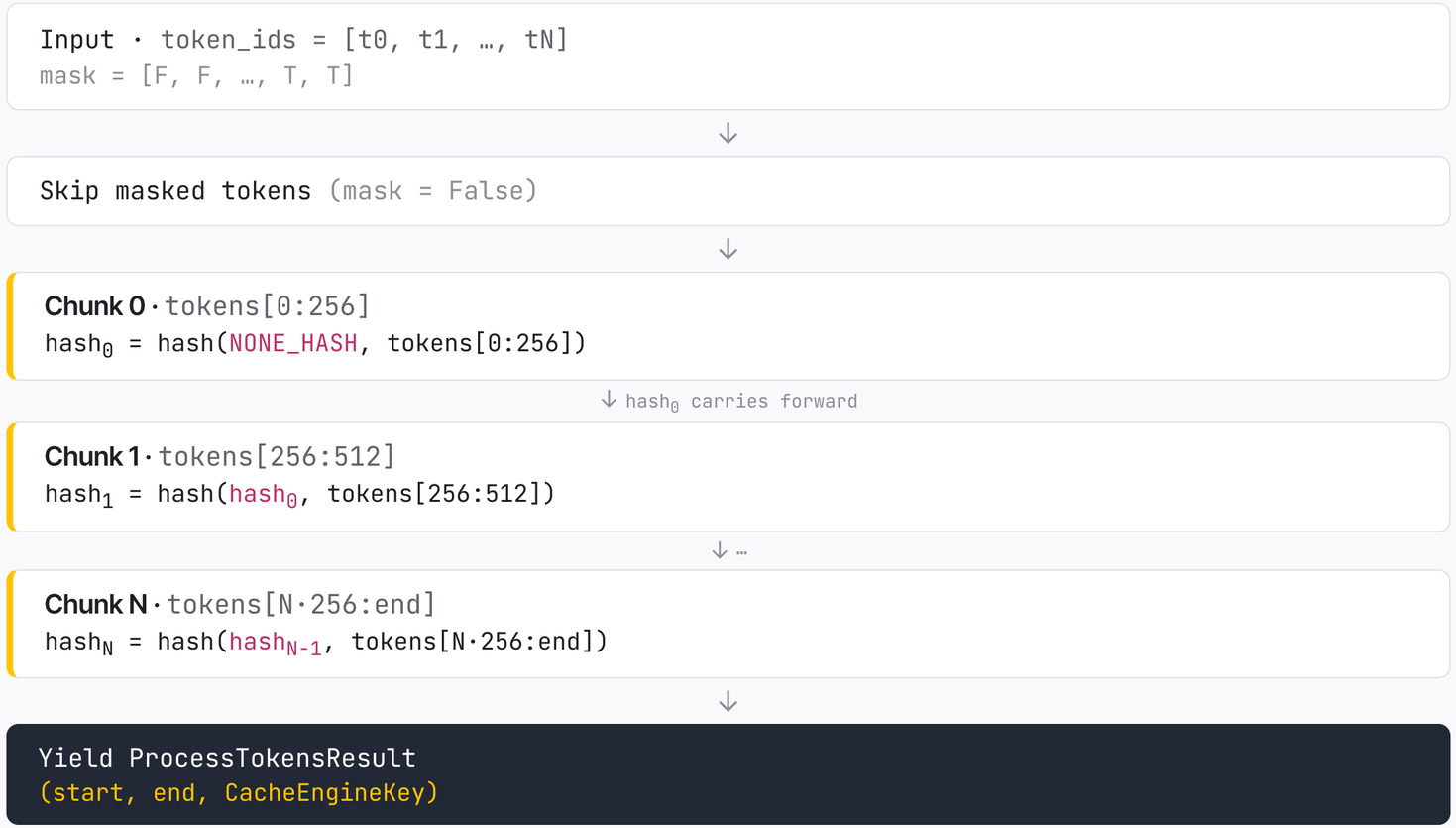
The **ChunkedTokenDatabase** splits a token sequence into fixed **256-token chunks** and hashes each into a `CacheEngineKey`.

`Prefix-hash chain`

Each chunk's hash folds in **all previous chunks** — so an identical prefix always yields an identical key, reusable on **any engine**.

`Mask-aware`

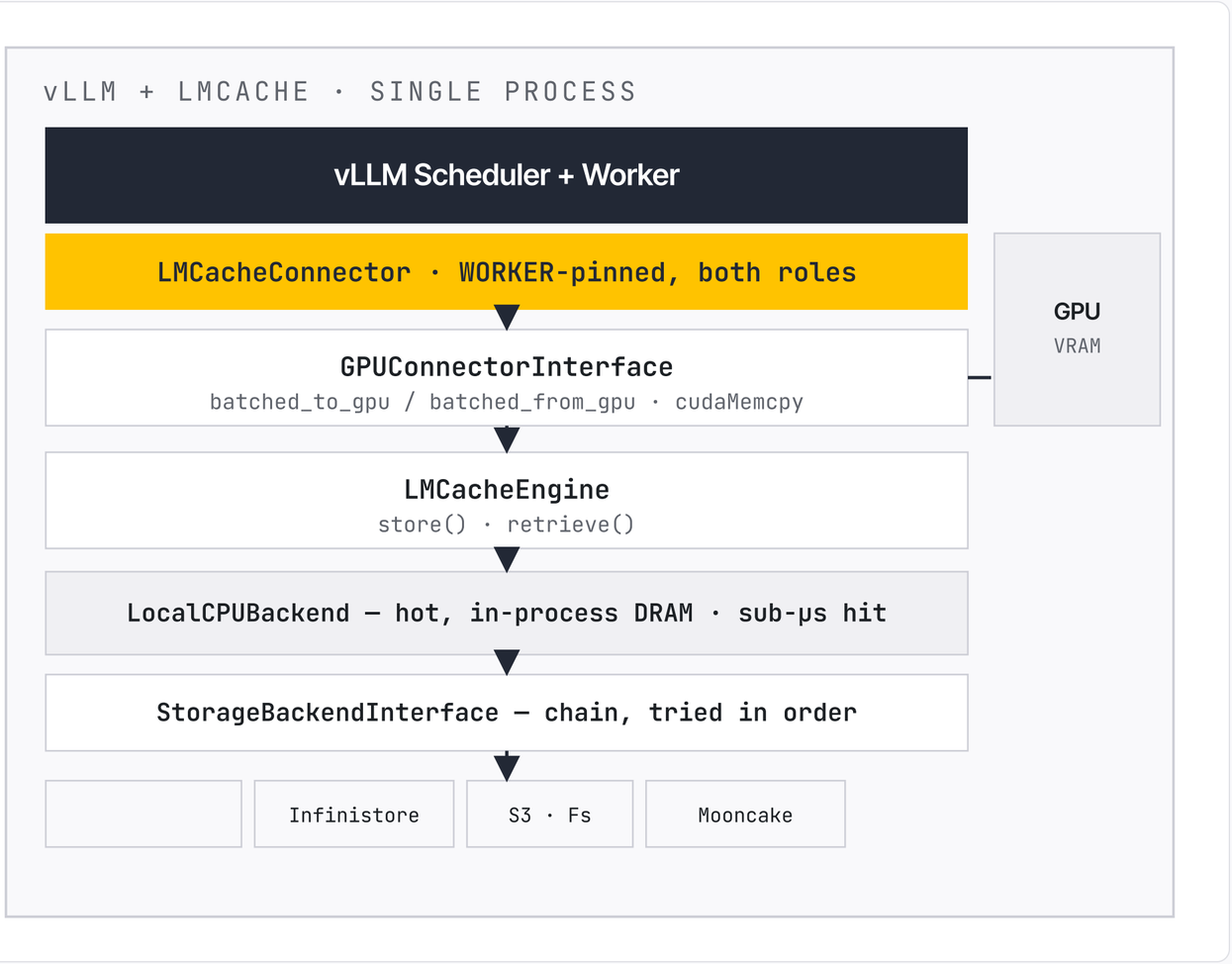
Tokens with `mask = False` are skipped before chunking. Variable-size segments are also supported — same hashing.



© 2026 XCENA Inc.

IN-PROCESS MODE · IN-PROCESS CONNECTOR

In-Process Mode: LMCache in vLLM



Same process

No IPC — but LMCache shares CPU & memory with the vLLM workers.

Synchronous retrieve

Blocks the scheduler step until the data is ready — no async prefetch window.

No central L1Manager

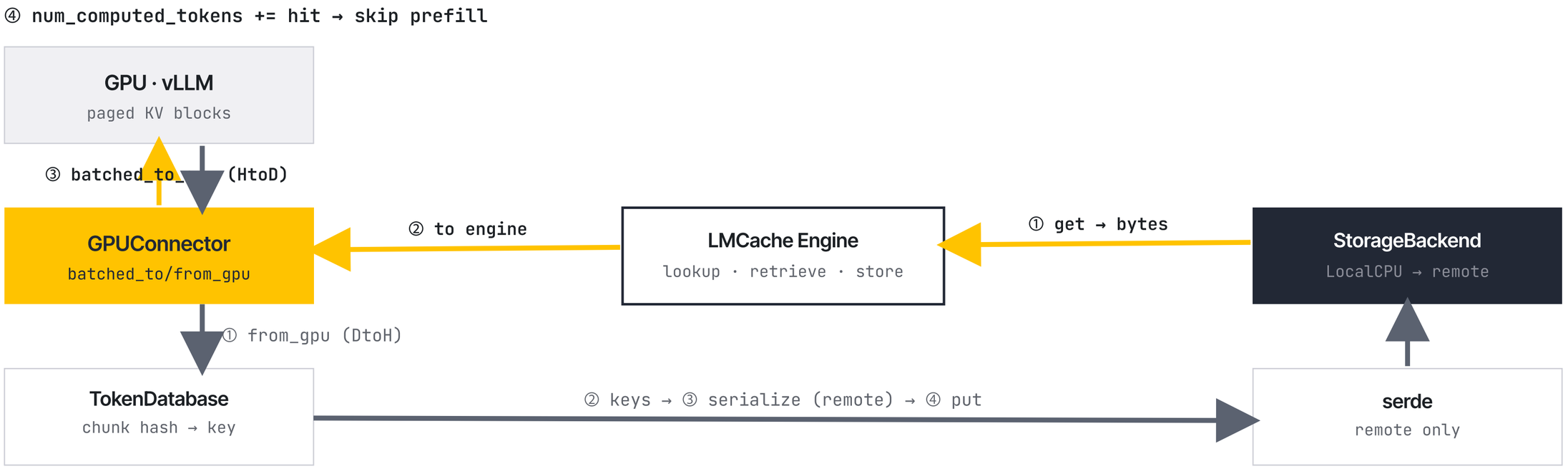
Each StorageBackend owns and allocates its own CPU buffers.

© 2026 XCENA Inc.

IN-PROCESS MODE · THE TWO DATA PATHS

retrieve() & store(), Step by Step

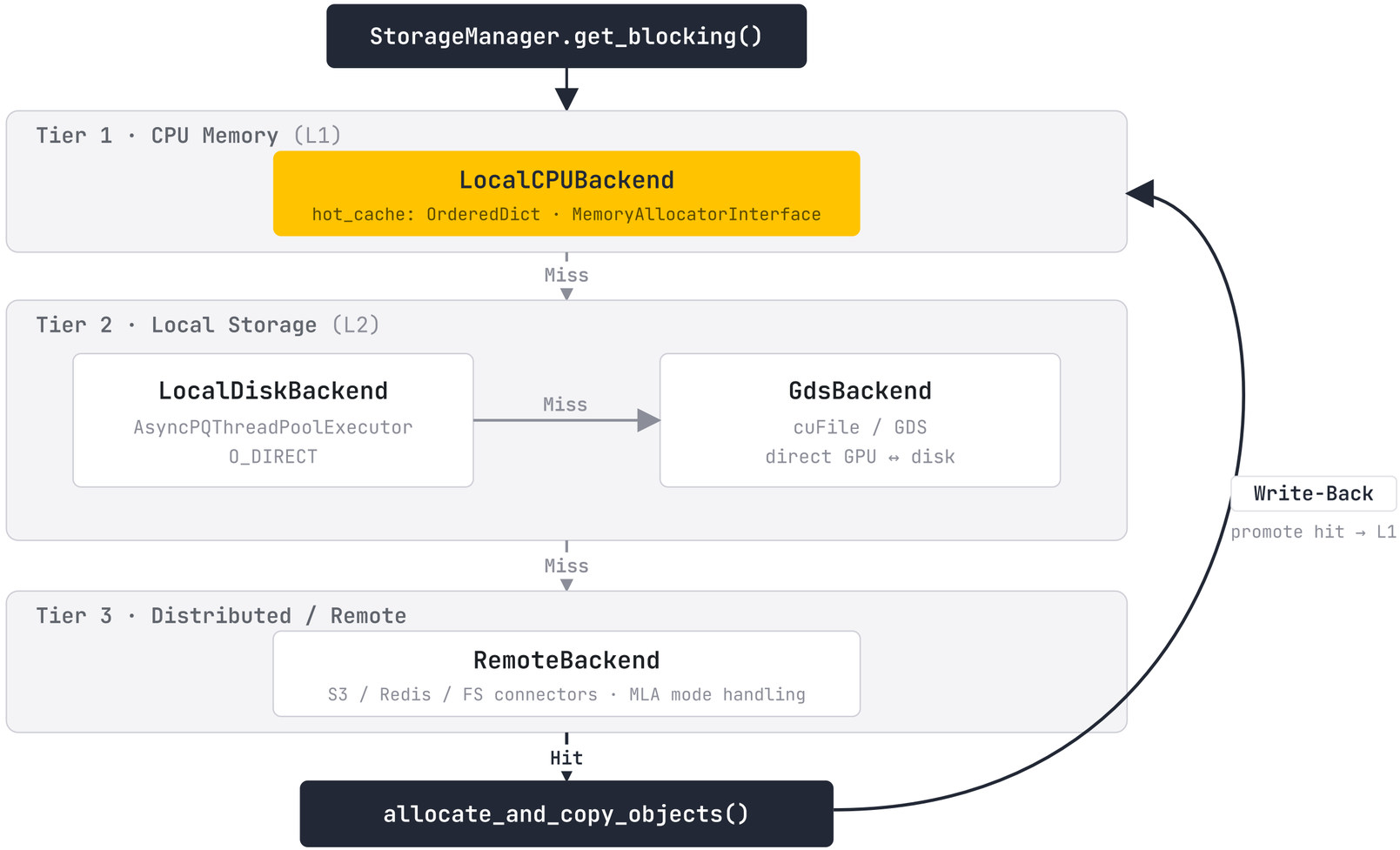
retrieve · before forward (GPU ← cache) store · after forward (GPU → cache)



Local hit = fast path (no serde). Both directions pivot on **GPUConnector** + **StorageBackend chain** — synchronous, in-process.

© 2026 XCENA Inc.

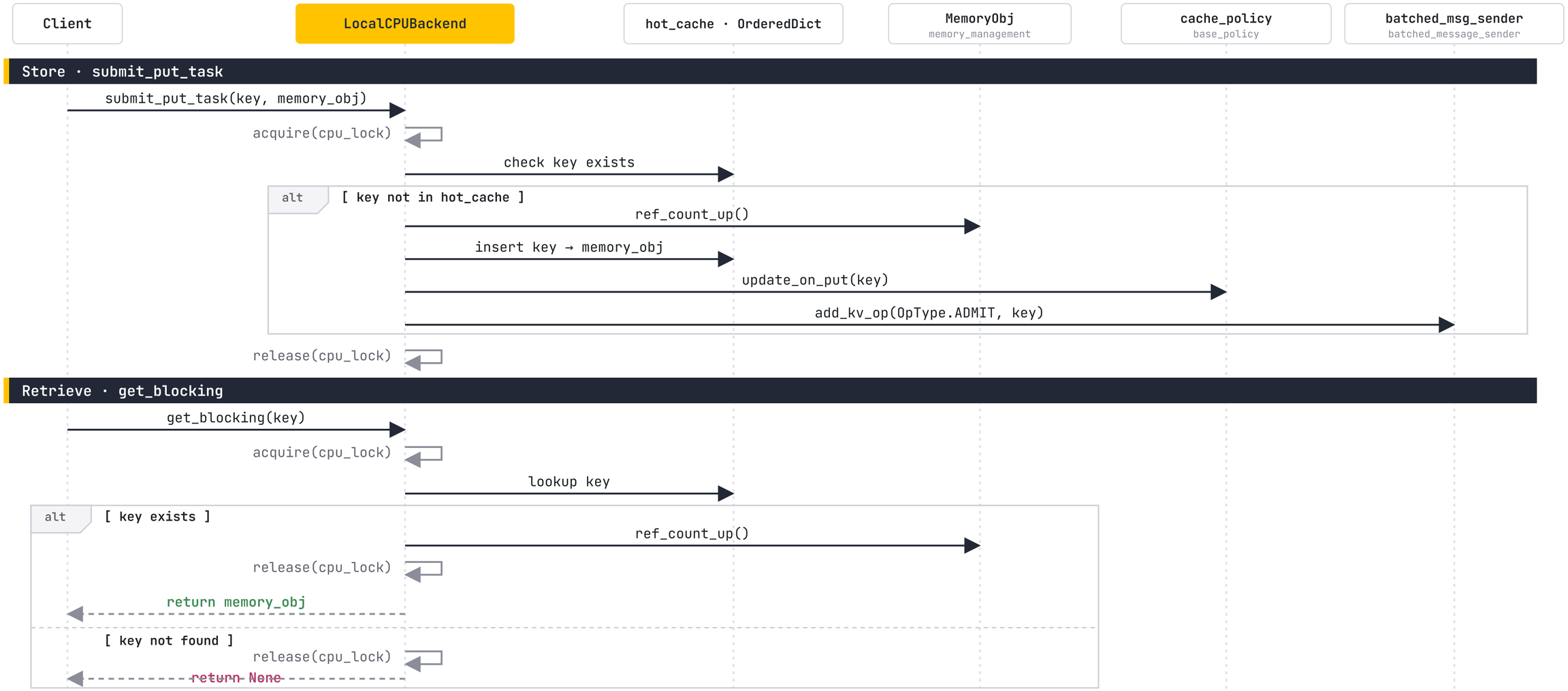
get_blocking(): Walk the Tiers, Promote on Hit



Tiers are tried in order — **L1 DRAM → L2 disk / GDS → L3 remote**. The first hit copies the objects and writes them back up into the hot L1 tier, so the hierarchy warms itself.

© 2026 XCENA Inc.

submit_put_task() & get_blocking(), Under One Lock



One **cpu_lock** serializes every mutation · **ref counting** keeps the zero-copy buffer alive while it is shared · ADMIT events tell the controller what now lives in L1

© 2026 XCENA Inc.

Why Run LMCache as Its Own Process

Process isolation
LMCache and vLLM run in separate processes — a cache-side crash never takes down the inference engine.

No GIL contention
LMCache Python work — hashing, memory mgmt, L2 I/O — stops competing with vLLM inference threads.

Shared cache across pods
Many vLLM instances on a node share one L1 cache → maximum KV reuse.

Independent scaling
Size CPU cache memory separately from GPU inference memory.

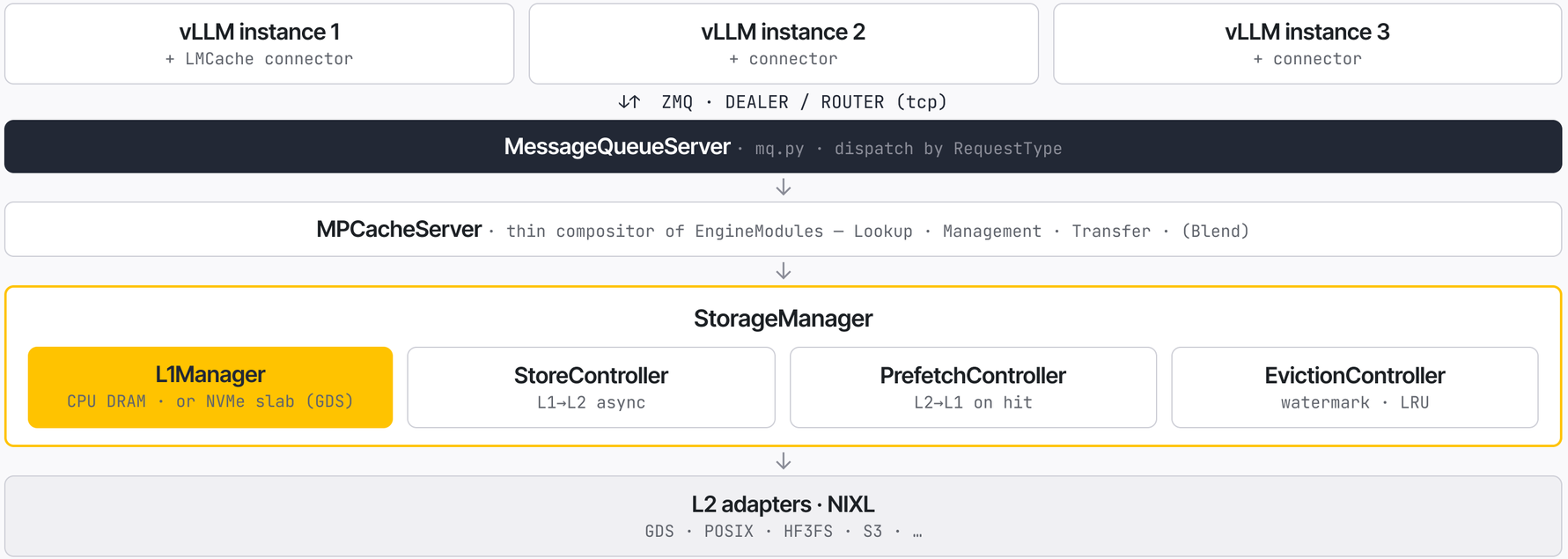
Multi-tier L1 + L2
L1 in CPU DRAM (or an NVMe slab via GDS) backed by persistent L2 over NIXL.

Built-in observability
Prometheus metrics and an OTel telemetry event system out of the box.

© 2026 XCENA Inc.

MULTI-PROCESS MODE · ARCHITECTURE

One Server, Many vLLM Instances



© 2026 XCENA Inc.

MULTI-PROCESS MODE · ZMQ PROTOCOL

Talking to the Server: RequestTypes

Register KV cache — SYNC

`REGISTER_KV_CACHE` `UNREGISTER_KV_CACHE`

Transfer · GPU ↔ L1 — BLOCKING

`STORE` `RETRIEVE`

Lookup & prefetch — BLOCKING

`LOOKUP` `QUERY_PREFETCH_STATUS` `QUERY_PREFETCH_LOOKUP_HITS` `FREE_LOOKUP_LOCKS`

Session & admin — mixed

`END_SESSION` `CLEAR` `GET_CHUNK_SIZE` `PING`

SYNC — runs in the ZMQ loop, fast & non-blocking **BLOCKING** — thread pool · GPU copies or I/O

© 2026 XCENA Inc.

MULTI-PROCESS MODE · BACKGROUND CONTROLLERS

Three Controllers Keep Tiers in Sync

StoreController

`store_controller.py`

Event-driven (eventfd poll). New L1 objects → async **L1 → L2** push to each adapter, per StorePolicy.

PrefetchController

`prefetch_controller.py`

On LOOKUP, keys missing from L1 → query L2 adapters and load found data **L2 → L1**.

EvictionController

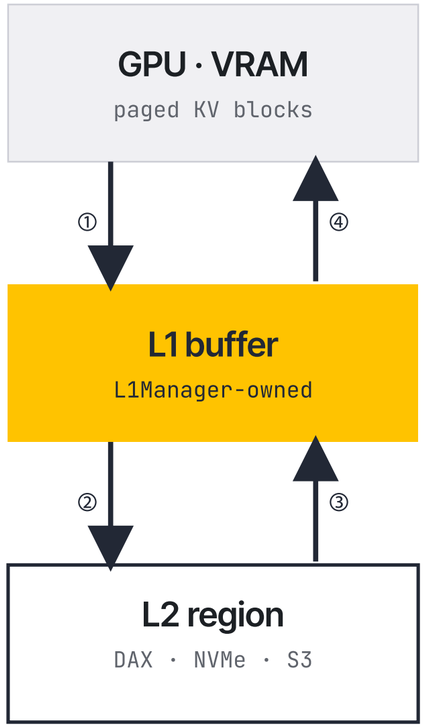
`eviction_controller.py`

Watermark-triggered. Evicts by **LRU / IsolatedLRU / noop**; IsolatedLRU enforces per-`cache_salt` quotas.

© 2026 XCENA Inc.

MULTI-PROCESS · COPY LEDGER

Four Copies, End to End



1	GPU → L1 · store	`cudaMemcpy DtoH`
2	L1 → L2 · Copy A	`ctypes.memmove`
3	L2 → L1 · Copy B	`ctypes.memmove`
4	L1 → GPU · load	`cudaMemcpy HtoD`

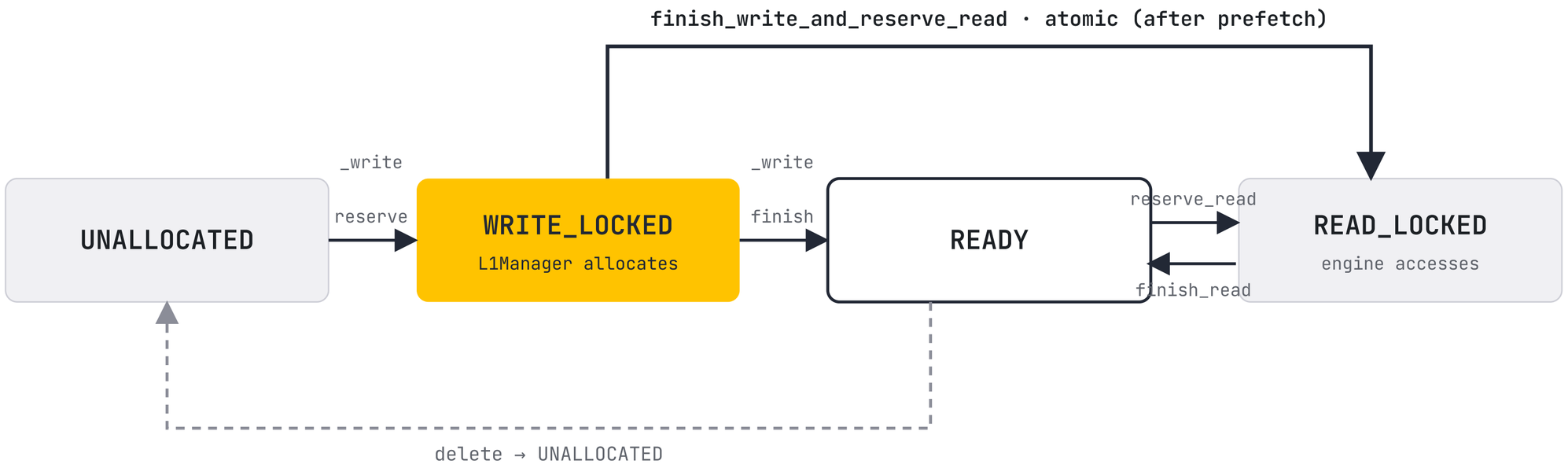
L1Manager owns the destination before L2 copies into it.

L2 is a writer-for-hire — never allocates. OOM is caught at reserve, not mid-I/O.

© 2026 XCENA Inc.

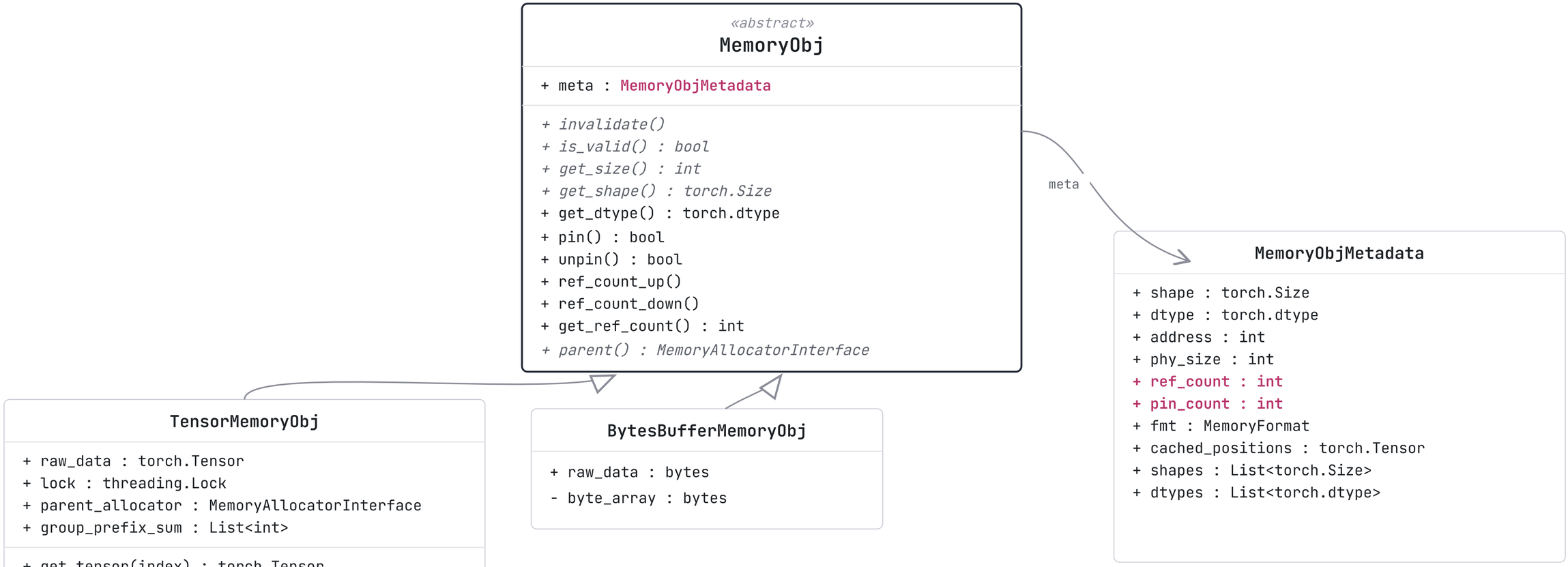
MULTI-PROCESS · L1MANAGER · TTLLOCK

The L1 Buffer State Machine



© 2026 XCENA Inc.

MemoryObj — One Interface, Many Formats



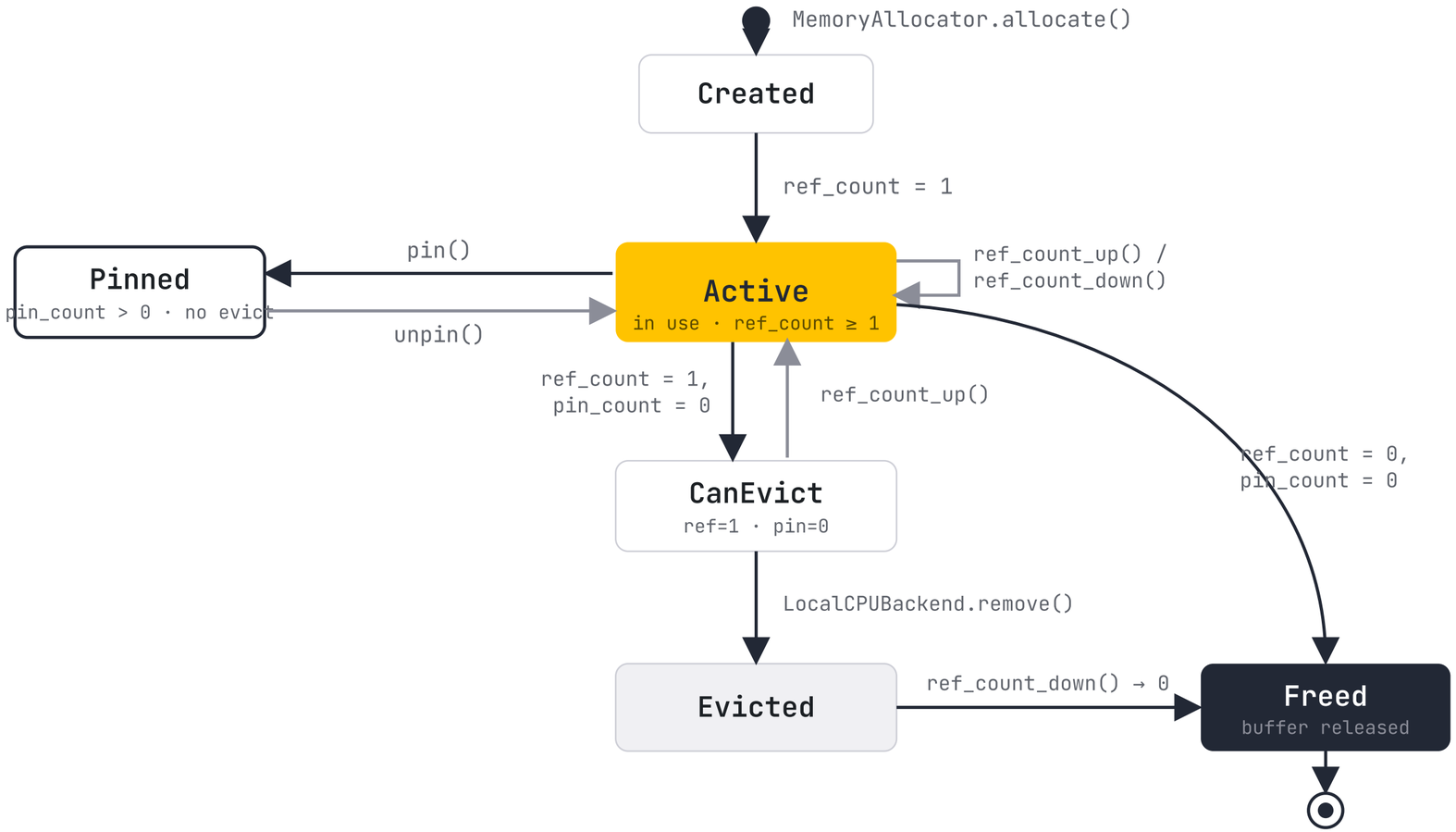
One contract — ref-count · pin · metadata — over two concrete formats: **TensorMemoryObj** (in-memory) and **BytesBufferMemoryObj** (serialized). The live counters sit in **meta**.

© 2026 XCENA Inc.

A MemoryObj's Life: ref_count + pin_count

ref_count
Active uses in the **code space** — a worker processing it, or a pending storage task. Hits 0 → buffer can be freed.

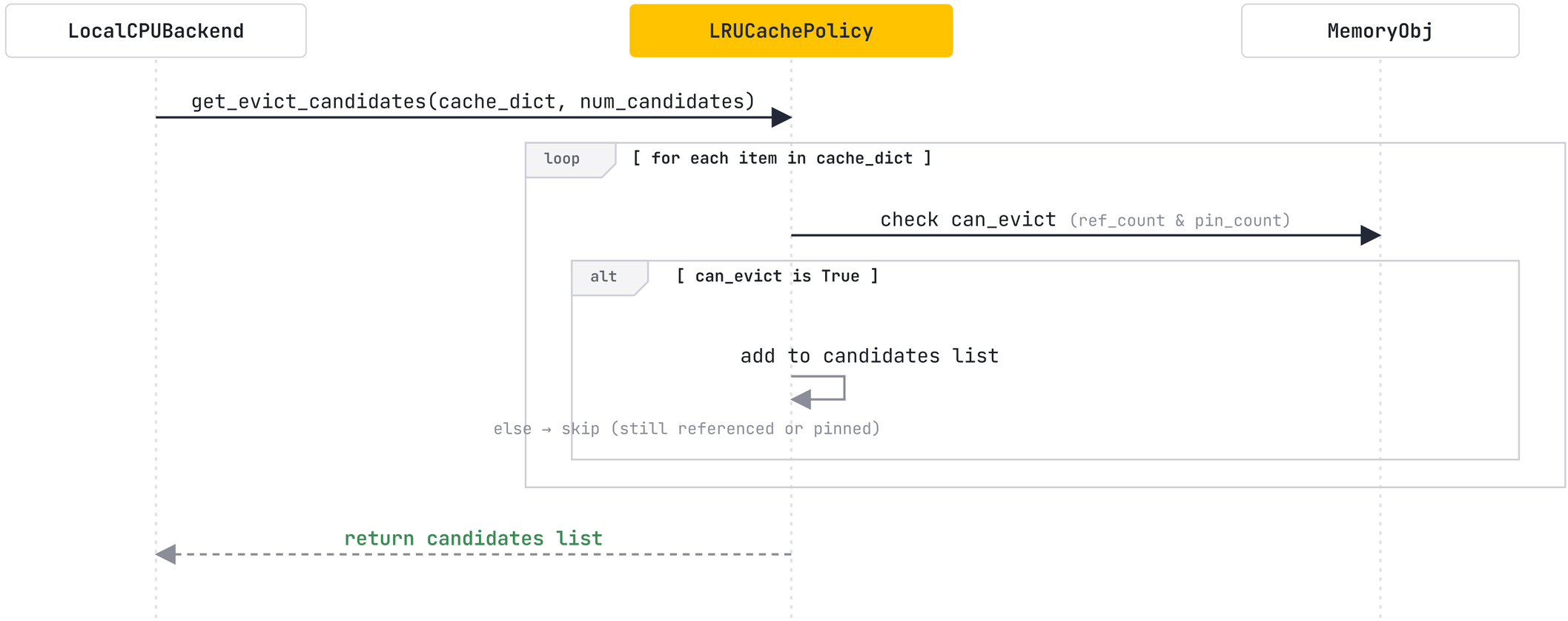
pin_count
Guards against **eviction** during critical ops — a lookup, or active serving. Set by pin() / cleared by unpin().



Dual control: **ref_count** tracks code-space uses, **pin_count** blocks eviction. Nothing is freed while either is held — the basis of safe zero-copy sharing.

© 2026 XCENA Inc.

Who to Evict? The Policy Decides

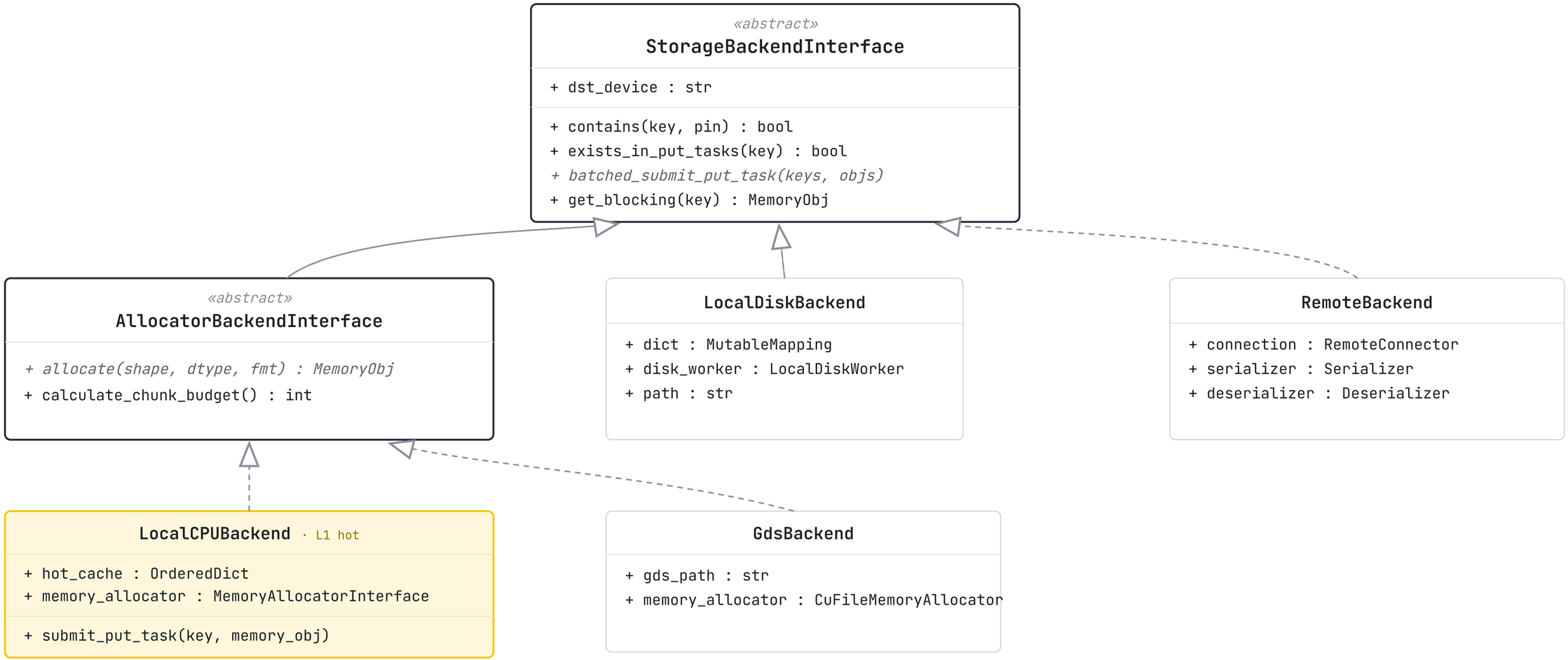


The backend owns the memory + **hot_cache** mapping; the **cache_policy** owns the eviction decision — swap LRU for another policy without touching the backend.

local_cpu_backend.py : 59-60

© 2026 XCENA Inc.

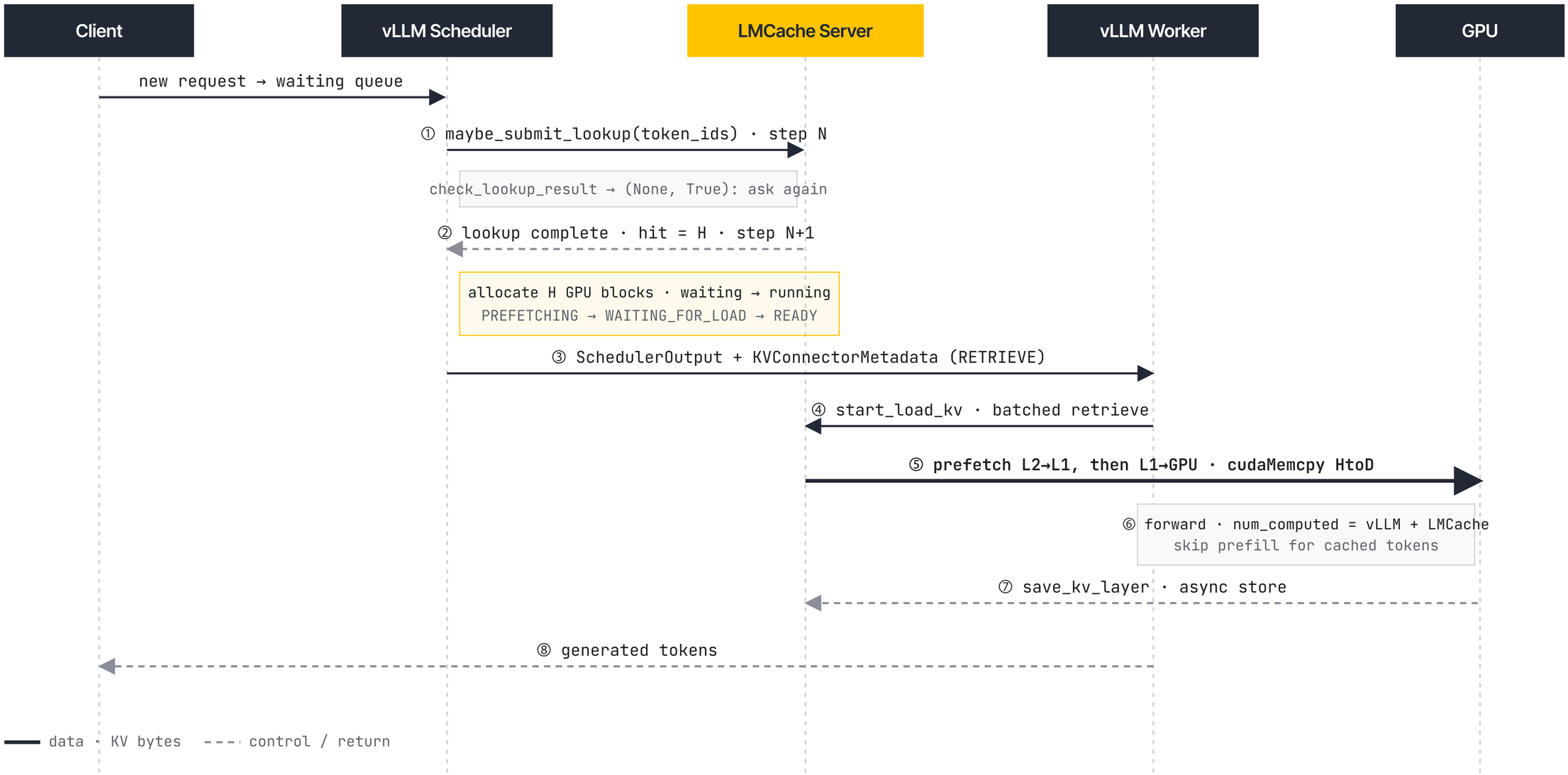
One Backend Contract, Many Tiers



Allocating tiers (**LocalCPU**, **Gds**) own buffers via AllocatorBackendInterface; serializing tiers (**LocalDisk**, **Remote**) implement the interface directly. The StorageManager treats them as one chain.

© 2026 XCENA Inc.

One Request, End to End



© 2026 XCENA Inc.

EVALUATION

Does It Actually Pay Off?

The LMCache paper's measured results — vs basic vLLM, vLLM CPU offloading, and two commercial services, on 8×H100.

© 2026 XCENA Inc.

EVALUATION · CPU-OFFLOAD SCENARIO · 8×H100

1.9–8.1× Faster TTFT, up to 14× Throughput

1.9–8.1×
smaller TTFT
vs strongest baseline @ QPS 1

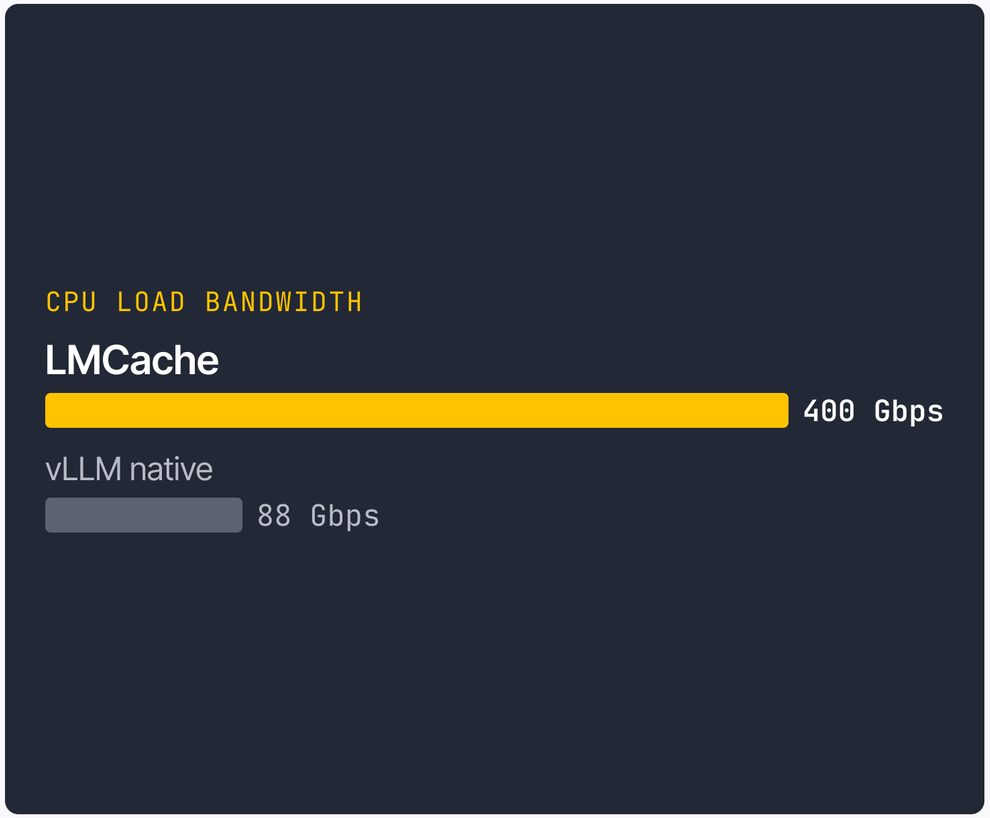
2.3–14×
higher throughput
same TTFT · 5 models

7–92%
smaller ITL
vs best baseline @ QPS 1

Why it wins

- CPU memory holds far more KV → **higher hit ratio** than GPU-only prefix cache
- loads KV at **chunk level with CUDA kernels**, not per-layer 16-token pieces

baselines: basic vLLM · vLLM CPU offload · 2 commercial services



Paper-reported · LMCache v0.3.6 vs vLLM v0.10.2 / v0.11.0 (arXiv:2510.09665, Fig 8 · Table 5).

© 2026 XCENA Inc.

EVALUATION · FIGURE 8 · CPU-OFFLOAD, 5 MODELS

LMCache Stays Flat as Load Climbs

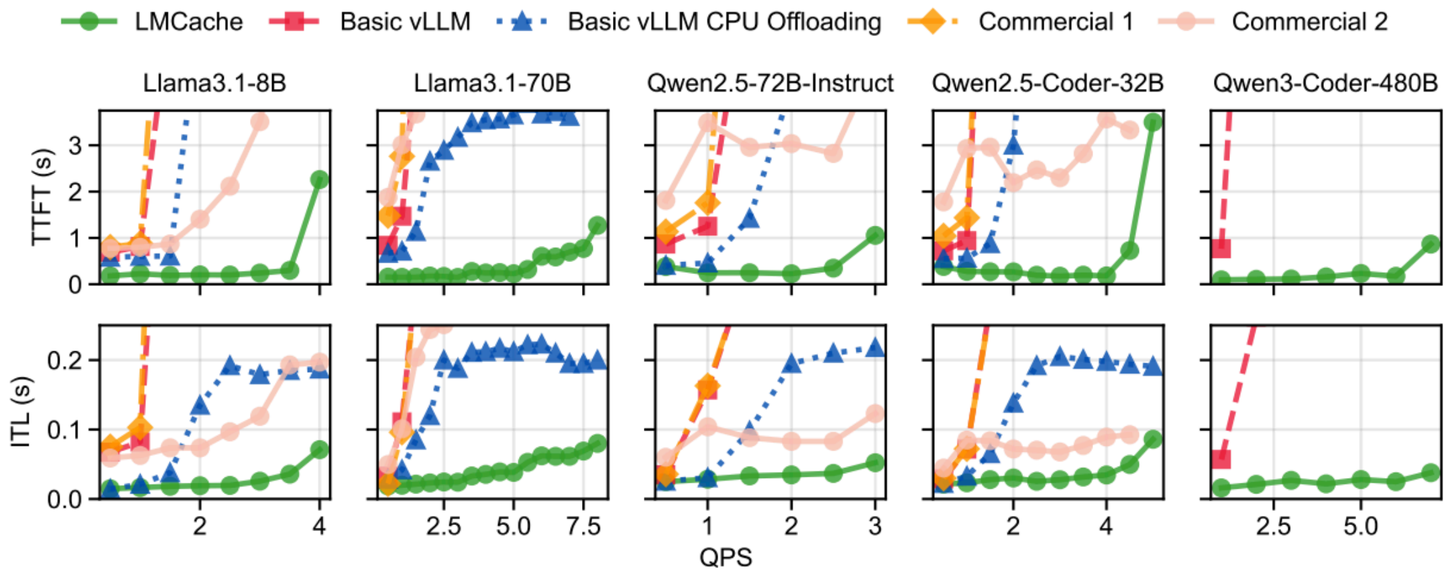


Figure 8: *Compared to basic vLLM, basic vLLM CPU offloading, and two commercial alternatives,* LMCache *has 1.9–8.1× smaller TTFT, and supports 2.3–14× higher inference throughput. Basic vLLM CPU offloading fails to run on* `Qwen3-Coder-480B`*, and commercial alternatives do not have the option to deploy* `Qwen3-Coder-480B`.

LMCache — flat & low baselines climb steeply with load (QPS →) **1.9–8.1× TTFT · 2.3–14× throughput**

Reproduced from LMCache paper, Figure 8 (arXiv:2510.09665).

© 2026 XCENA Inc.

EVALUATION · CONSISTENT ACROSS FOUR SCENARIOS

Holds Across Every Deployment

① CPU OFFLOAD

8.1×

TTFT ↓ (max)

multi-round doc QA · 14× throughput

② REAL TRACE

3.7–6.8×

TTFT ↓

company F/G dist · 19–58% ITL ↓

③ REMOTE STORE

1.3–3×

throughput ↑

15 Gbps link · bigger cache wins

④ PD DISAGG

1.53–1.84×

mean TTFT ↓

chunk buffer > page-by-page NIXL

Paper-reported · arXiv:2510.09665 (Fig 9-12). All vs basic vLLM with GPU prefix caching.

© 2026 XCENA Inc.

REAL-WORLD ADOPTION

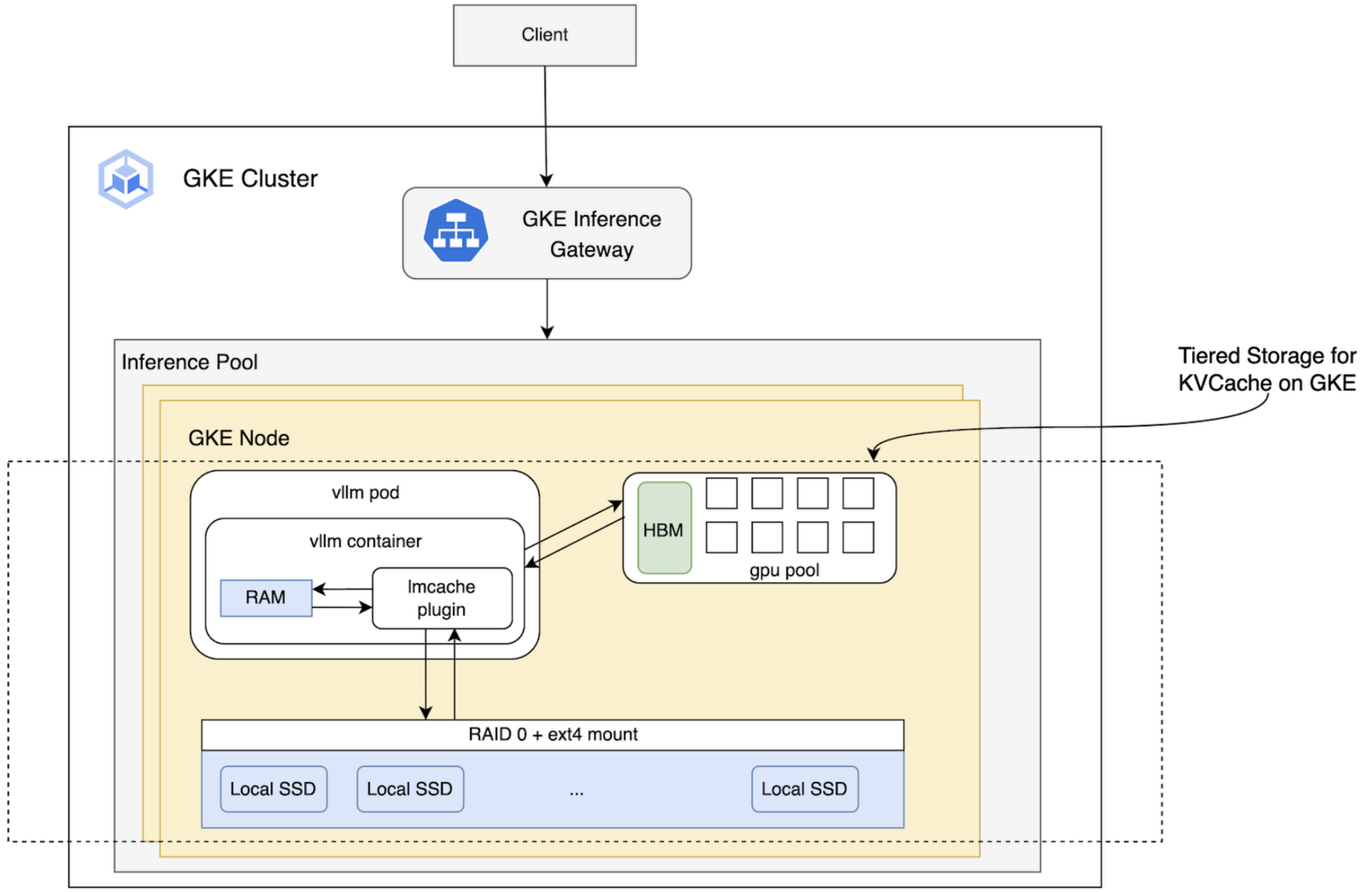
LMCache in Production

The tiered, offloaded KV-cache approach is shipping — across cloud, enterprise, and storage vendors.

VENDOR	STACK	BACKEND / TIER	MAIN BENEFIT
IBM	`llm-d + LMCache + IBM Storage Scale`	distributed file storage	cost & TTFT economics
Google Cloud	`vLLM + LMCache on GKE`	HBM + CPU RAM + Local SSD	long-context tiered performance
Dell	`vLLM + LMCache + NVIDIA NIXL`	PowerScale / ObjectScale	external offload, TTFT acceleration

© 2026 XCENA Inc.

Node-Local Tiered Cache on GKE



three node-local tiers

HBM · 640 Gi

CPU RAM · 1 Ti

Local SSD · 5 Ti · RAID-0

8×H100 A3-mega · Llama-3.3-70B · shared-prefix, 1K–100K prompts

Source: Google Cloud — tiered KV cache on GKE (LMCache v0.3.3).

© 2026 XCENA Inc.

GOOGLE CLOUD · GKE · EXPERIMENT SETUP

Tiered KV Cache on GKE — Setup

hardware	8× NVIDIA H100 80GB · A3 mega
model	Llama-3.3-70B-Instruct
LMCache	v0.3.3
benchmark	SGLang `bench_serving`

HBM	640 Gi
CPU RAM	1 Ti
Local SSD	5 Ti

three cache-tier configurations

① HBM only
baseline — GPU-local cache

② HBM + CPU RAM
warm tier in host memory

③ HBM + CPU RAM + Local SSD
cold tier on node-local SSD

Vendor-reported setup · Google Cloud (GKE) · node-local tiering, no remote storage.

© 2026 XCENA Inc.

GOOGLE CLOUD · GKE · EXPERIMENT WORKLOAD

A Shared Prompt, Scaled 1K → 100K

one batch = 20 requests sharing a prompt

shared system prompt · 1K–100K tok (the reuse target)

↓ shared across

256 in | 512 out

256 in | 512 out

256 in | 512 out

⋮

× 20

Only the **256-token tail** differs per request → the long prompt is pure shared-prefix reuse.

each prompt length stands in for a scenario

1K–5K	**High-fidelity personas**	complex, detailed instructions
10K	**Average user prompt**	small RAG payload · web page / article
50K	**Prompt stuffing**	large context packed into the prompt
100K	**Long-book equivalent**	book-length context window

Vendor-reported workload · Google Cloud (GKE) · 20-request shared-context batches, 256-tok input / 512-tok output.

© 2026 XCENA Inc.

When Tiering Helps

`cache fits in HBM` → **HBM only — extra tiers don't help**

`exceeds HBM, fits CPU RAM` → **HBM + CPU RAM is best**

`very long / larger cache` → **add Local SSD**

STRONGEST GAINS

50K · 100K

token contexts — where cache demand blows past HBM.

Lesson: **tiering helps when cache demand exceeds HBM** — not before.

Vendor-reported results · Google Cloud.

© 2026 XCENA Inc.

GOOGLE CLOUD · TEST 2 · CACHE FITS HBM + CPU RAM (4M TOK)

Tiering Wins as Prompts Grow

PROMPT	BEST SETUP	TTFT	THROUGHPUT	E2E LATENCY
1K	HBM	0%	0%	0%
5K	HBM + CPU RAM	-18%	+16%	-14%
10K	HBM + CPU RAM	-44%	+50%	-33%
50K	+ Local SSD	-68%	+179%	-64%
100K	+ Local SSD	-79%	+264%	-73%

@ 100K prompt: **TTFT –79% · throughput +264% · latency –73%** vs HBM-only.

Source: Google Cloud, Test 2 (% change vs HBM-only; green = better).

© 2026 XCENA Inc.

GOOGLE CLOUD · TEST 3 · SPILLS TO LOCAL SSD (13M TOK)

Deep Tier Helps Long Prompts

PROMPT	BEST SETUP	TTFT	THROUGHPUT	E2E LATENCY
1K	HBM + CPU RAM	+5%	+1%	-1%
5K	HBM + CPU RAM	-6%	+27%	-21%
10K	HBM + CPU RAM	+121%	+23%	-19%
50K	+ Local SSD	+48%	+69%	-41%
100K	+ Local SSD	-3%	+130%	-57%

Takeaway: tiering helps once cache outgrows HBM & scales with prompt length — but deploy only the tier you need (SSD can hurt small caches).

Source: Google Cloud, Test 3 (% vs HBM-only; throughput up at every length).

© 2026 XCENA Inc.

IBM · LLM-D + LMCACHE + IBM STORAGE SCALE

Inference Economics, Architecture-Backed

persistent · distributed · shared KVCache

llm-d · inference orchestrator

↓

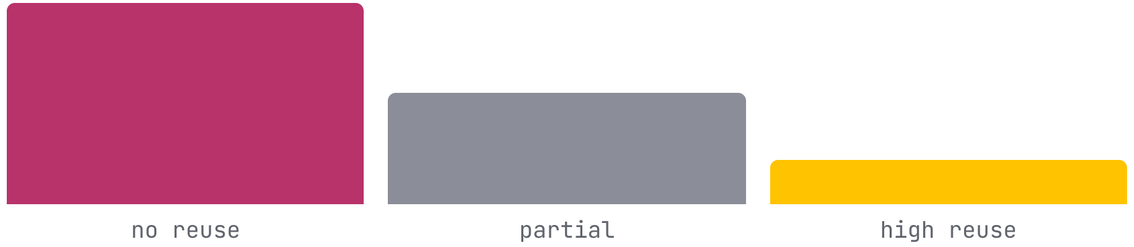
vLLM workers

↓

LMCache · KVCache reuse

↓

IBM Storage Scale · distributed store



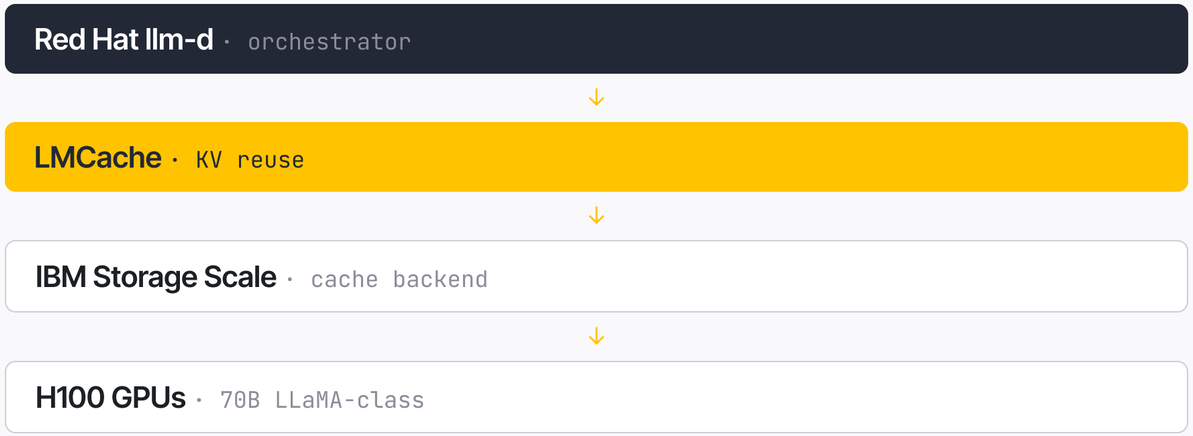
Every reused token is **GPU compute you don't pay for again** — the question becomes **latency per dollar**, not just latency.

Vendor-framed (IBM) · positioned for RAG, agents, tool-use, enterprise copilots.

© 2026 XCENA Inc.

Isolating the Cost of Reuse

the modeled system



Controlled workload

1 user · 1 query · fixed tokens/request · reuse swept **0 → 100%**. No concurrency (conservative).

Conservative cost model

GPU per-second (public H100) · CPU/server amortized · storage held constant → isolates reuse.

THE DRAM PROBLEM

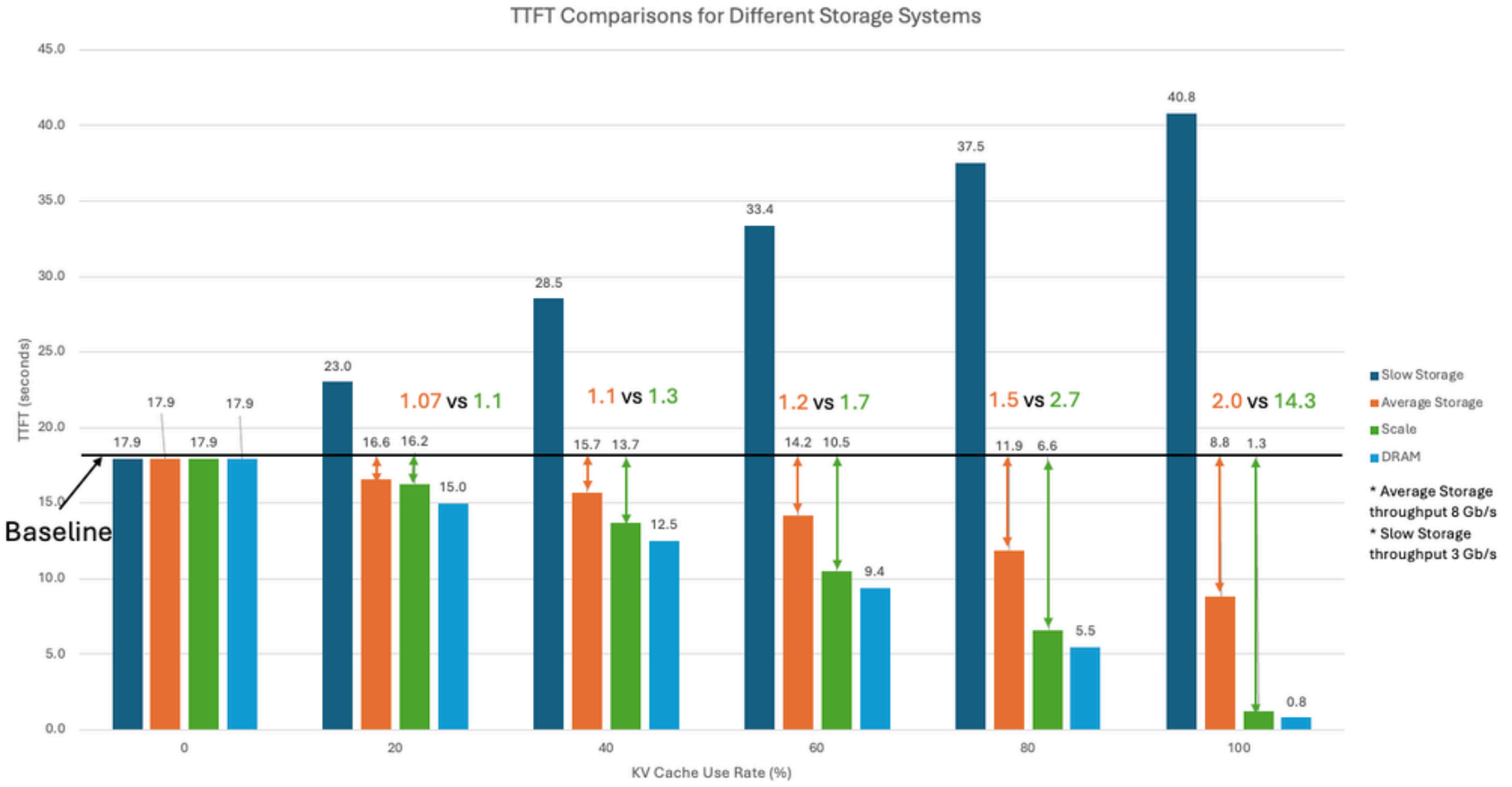
~$780K

100 TB DRAM KV (320 KB/token, 70B) → cost scales linearly. Why storage-backed wins.

Source: IBM — llm-d + LMCache + IBM Storage Scale (modeled, conservative).

© 2026 XCENA Inc.

Latency Falls Toward DRAM with Reuse



TTFT @ 100% reuse

17.9 s
no-reuse baseline

1.3 s
Storage Scale

0.8 s
DRAM (lower bound)

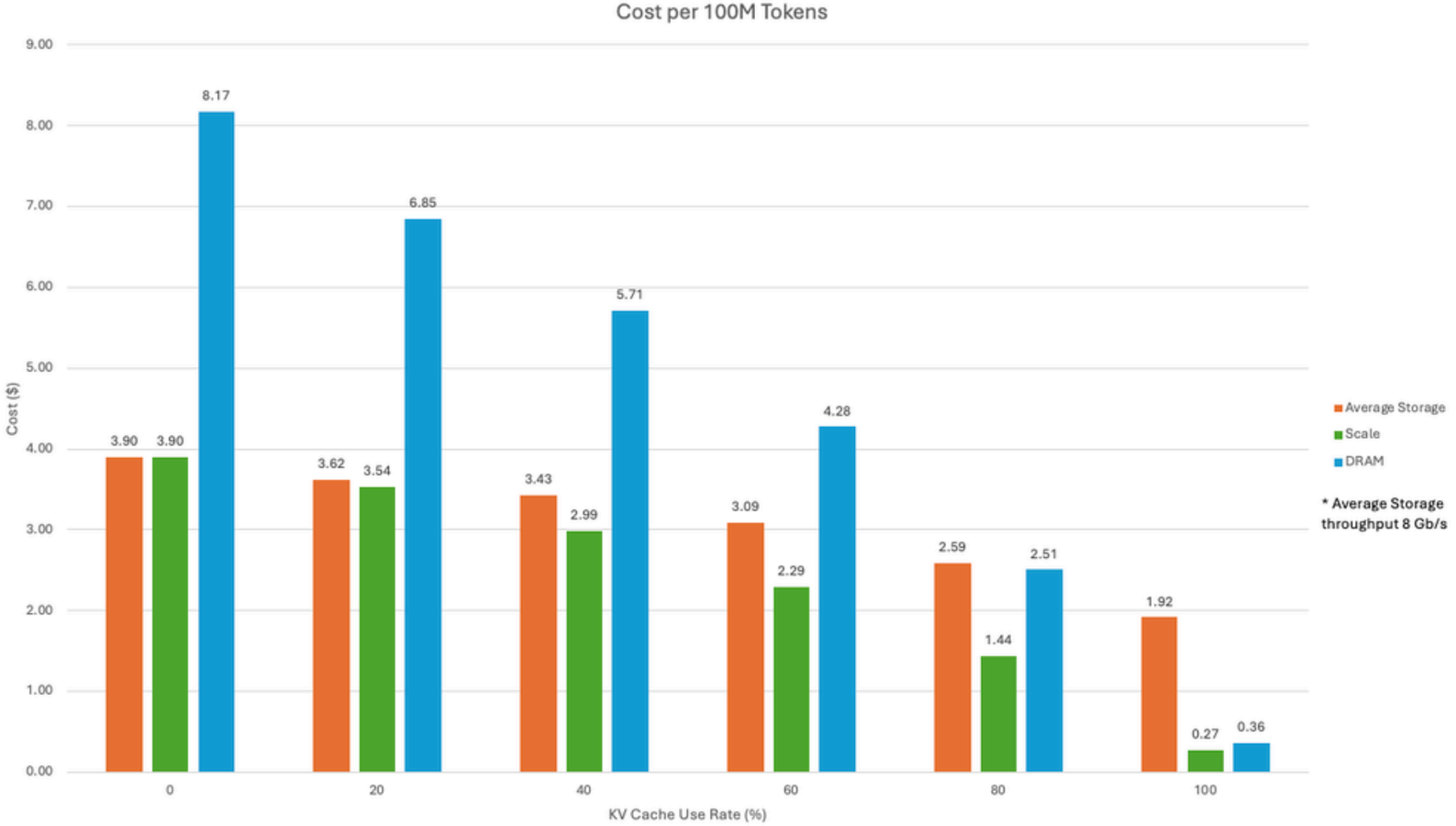
Why these curves — modeled **effective** throughput, not peak. KV reuse is latency-sensitive & concurrent → **5–10 GB/s** sustained (protocol overhead · metadata · contention · non-sequential access).

3 GB/s	8 GB/s	» 8 GB/s	lower bound
slow storage	average storage · midpoint	IBM Storage Scale · off critical path	DRAM · latency floor

Source: IBM, Figure 1 (modeled effective throughput; avg 8 GB/s, slow 3 GB/s).

© 2026 XCENA Inc.

Cost per Token Drops — Scale Cheapest



$ / 100M tok @ 100% reuse

$0.27
Storage Scale — cheapest

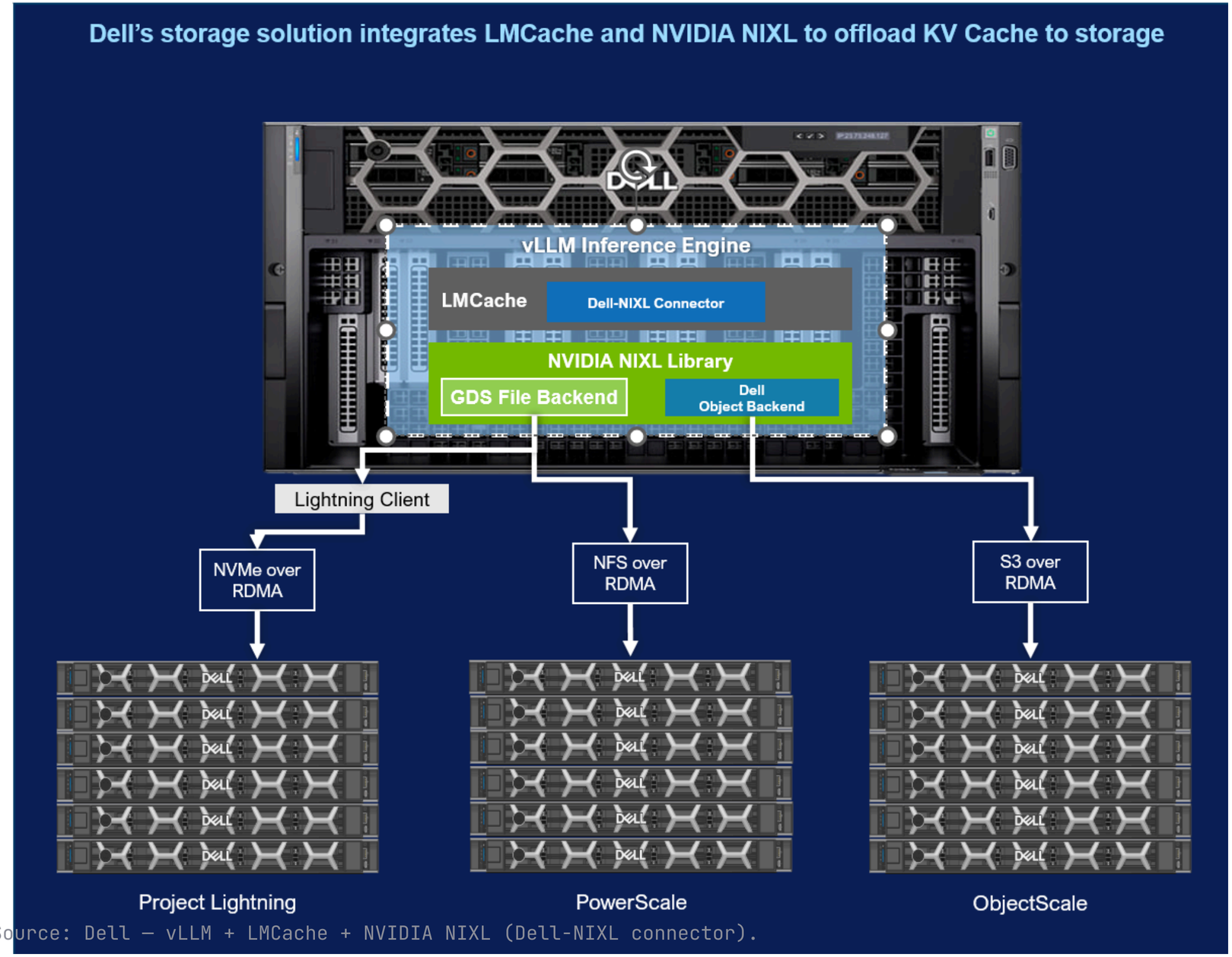
$0.36
DRAM

$1.92
average storage

near-DRAM latency **and** lowest cost — no tradeoff.

Source: IBM, Figure 2 (modeled; reuse 0→100%, cost down > 10×).

© 2026 XCENA Inc.



Source: Dell — vLLM + LMCache + NVIDIA NIXL (Dell-NIXL connector).

three RDMA paths

Project Lightning
NVMe-oF over RDMA

PowerScale
NFS over RDMA (GDS)

ObjectScale
S3 over RDMA (NIXL plugin)

one connector → **file + object**, RDMA to GPU.

© 2026 XCENA Inc.

Multi-Turn KV-Offload Benchmark

hardware	4× NVIDIA H100
model	Llama-3.3-70B · TP = 4
software	vLLM 0.10.2 · LMCache · Dell-NIXL 0.3.7 · NIXL 0.6.1
execution	Aggregated — prefill + decode, same GPUs
tool	LMbenchmark

storage backends · RDMA

PowerScale · Lightning
file · GDS

ObjectScale v4.2
object · RDMA S3

two cache configurations

BASE **Baseline**
KV cache entirely in HBM — standard vLLM

OFFLOAD **KV Cache Storage Offload**
vLLM + LMCache + NIXL + Dell connector → KV across GPU + shared storage. CPU memory excluded from the caching path.

multi-turn workload

80 concurrent users	10 rounds / user	1,000 sys prompt · shared
4,000 chat history · unique	200 response / turn	1 → 7 QPS swept

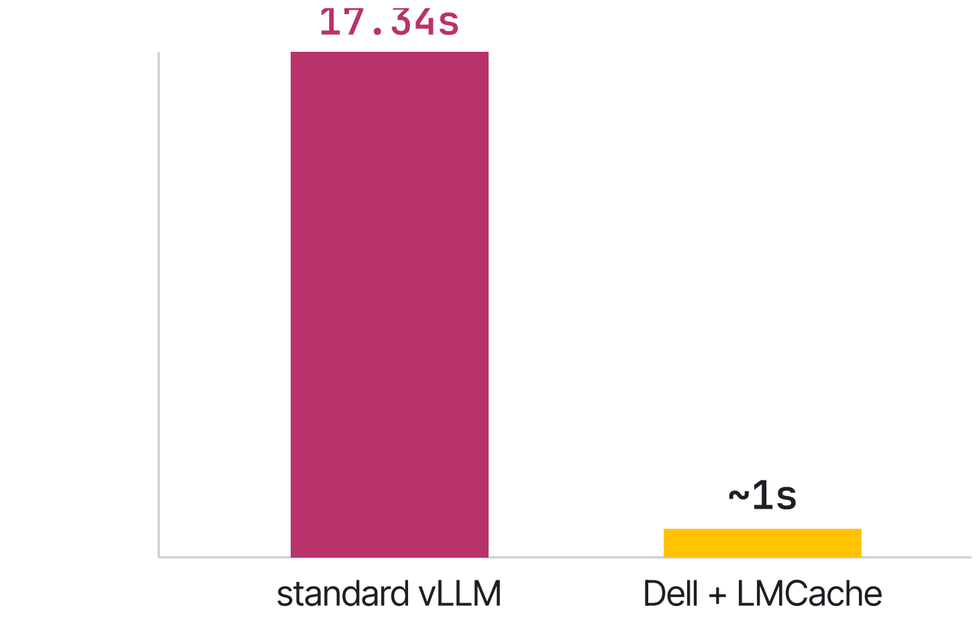
METRIC **TPS** tokens/sec across prefill + decode, vs QPS

Vendor-reported setup · Dell — vLLM + LMCache + NVIDIA Dynamo NIXL · PowerScale / ObjectScale / Project Lightning.

© 2026 XCENA Inc.

DELL · VLLM + LMCACHE + NVIDIA NIXL

Storage-Backed KVCache — 19× TTFT



PowerScale · ObjectScale · Project Lightning — all ≤ 1s

AT 131K CONTEXT

19× faster TTFT

17.34s → ~1s · retrieve vs recompute

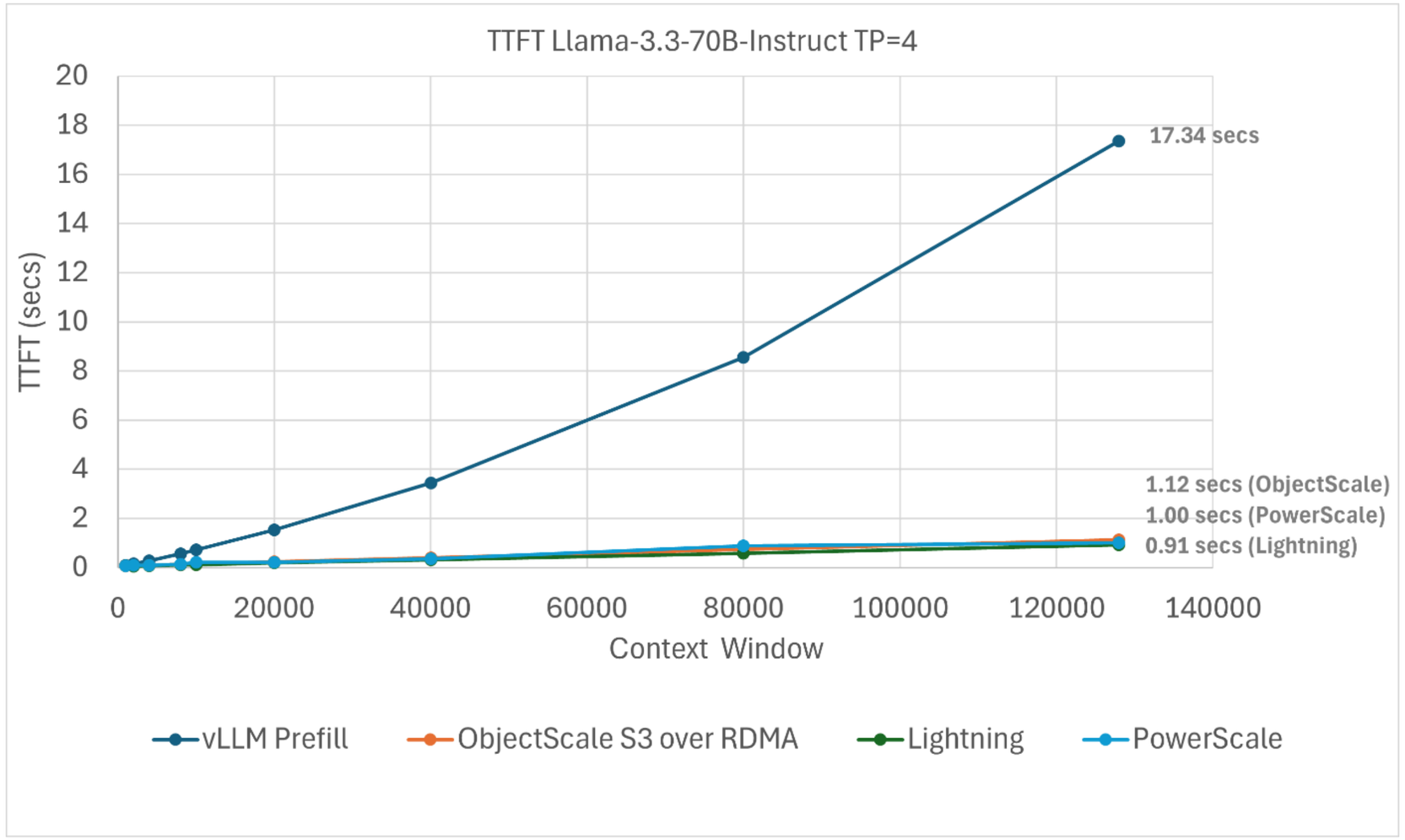
```
Llama-3.3-70B · TP=4 · 4×H100
vLLM 0.10.2 · LMCache Dell-NIXL 0.3.7 · NIXL 0.6.1
file: GDS / NFS-RDMA · object: RDMA-S3
```

TP=8: more GPU cuts vLLM only 17.3→12.3s; Dell stays **~1s**. Qwen3-32B: **14×** (11.82s→0.8s).

Vendor-reported results · Dell Technologies.

© 2026 XCENA Inc.

19× Faster TTFT at 131K Tokens



TTFT @ 131K context

17.34 s
standard vLLM

0.91 s
Lightning

1.00 s
PowerScale

1.12 s
ObjectScale

≈ 19×
faster

Source: Dell, TTFT benchmark · Llama-3.3-70B · TP=4 · 4×H100 · vLLM 0.10.2 · NIXL 0.6.1.

© 2026 XCENA Inc.

DELL · CASE STUDY · THE SCALING INSIGHT

Reuse Beats Brute-Force Compute

MORE GPUs · standard vLLM

4 → 8 H100 (TP=8)

17.3 → 12.3 s

2× the GPUs → only ~1.4× faster. Prefill still dominates.

KV REUSE · Dell storage

Unchanged by GPU scale

~1 s TP=4 or TP=8

no prefill to scale → order-of-magnitude win regardless of compute.

Adding compute without reuse buys a **modest** gain; **reusing cached KV** delivers an order of magnitude — the core argument for a KV cache layer.

Source: Dell — TP=8 scaling experiment, Llama-3.3-70B @ 131K.

© 2026 XCENA Inc.

LOOKING AHEAD · CXL

The Memory Wall, and CXL

Why a new memory tier exists — and what it unlocks for the KV cache.

© 2026 XCENA Inc.

A Memory Card on the PCIe Bus

01 CPU DRAM-like usage

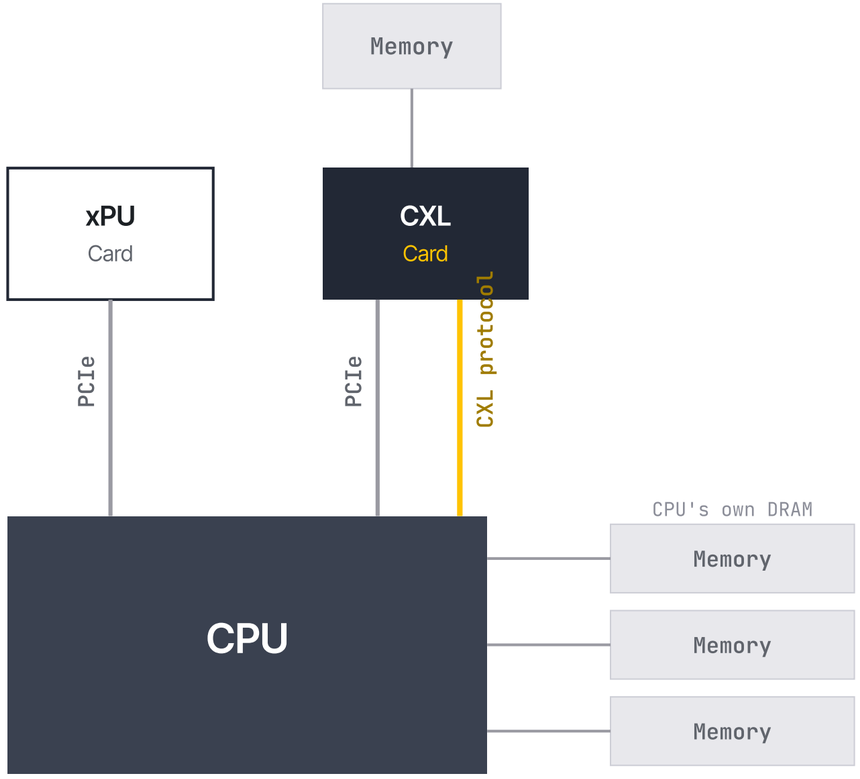
The CPU issues ordinary load/store to the card\u2019s memory over the CXL protocol — it behaves like extra DRAM.

02 xPU ↔ DMA

An accelerator (GPU / xPU) can DMA directly to and from that same memory over PCIe.

03 PCIe lanes

It plugs into a standard PCIe slot and rides commodity PCIe lanes — no special bus.



One card, two access paths: CPU load/store (CXL.mem) + accelerator DMA (PCIe).

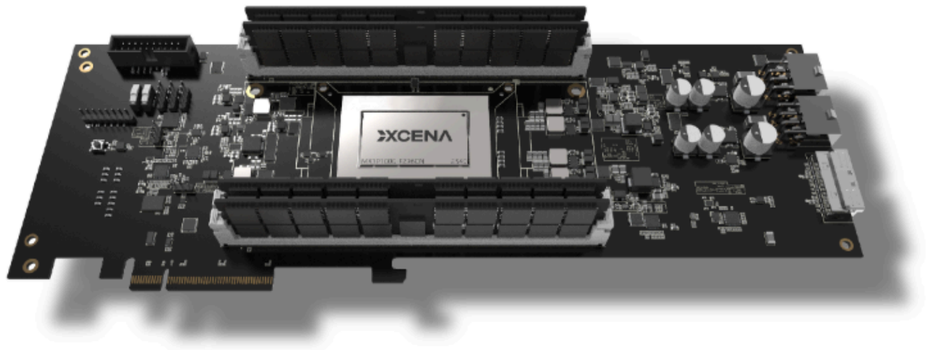
© 2026 XCENA Inc.

CXL · XCENA MEMORY EXPANSION · REAL HARDWARE

The CXL Card, in a Live Server

CXL Memory Expansion

INTERFACE	**CXL 3.2 · Type 3 HDM-DB with BI**
CAPACITY	**Up to 2 TB / card** (256 GB DIMM · 2 DPC)



XCENA CXL memory expansion card · CXL 3.2 Type 3 (memory device).

6× CXL cards beside NVIDIA GPUs — the physical home of Maru's shared KV pool.

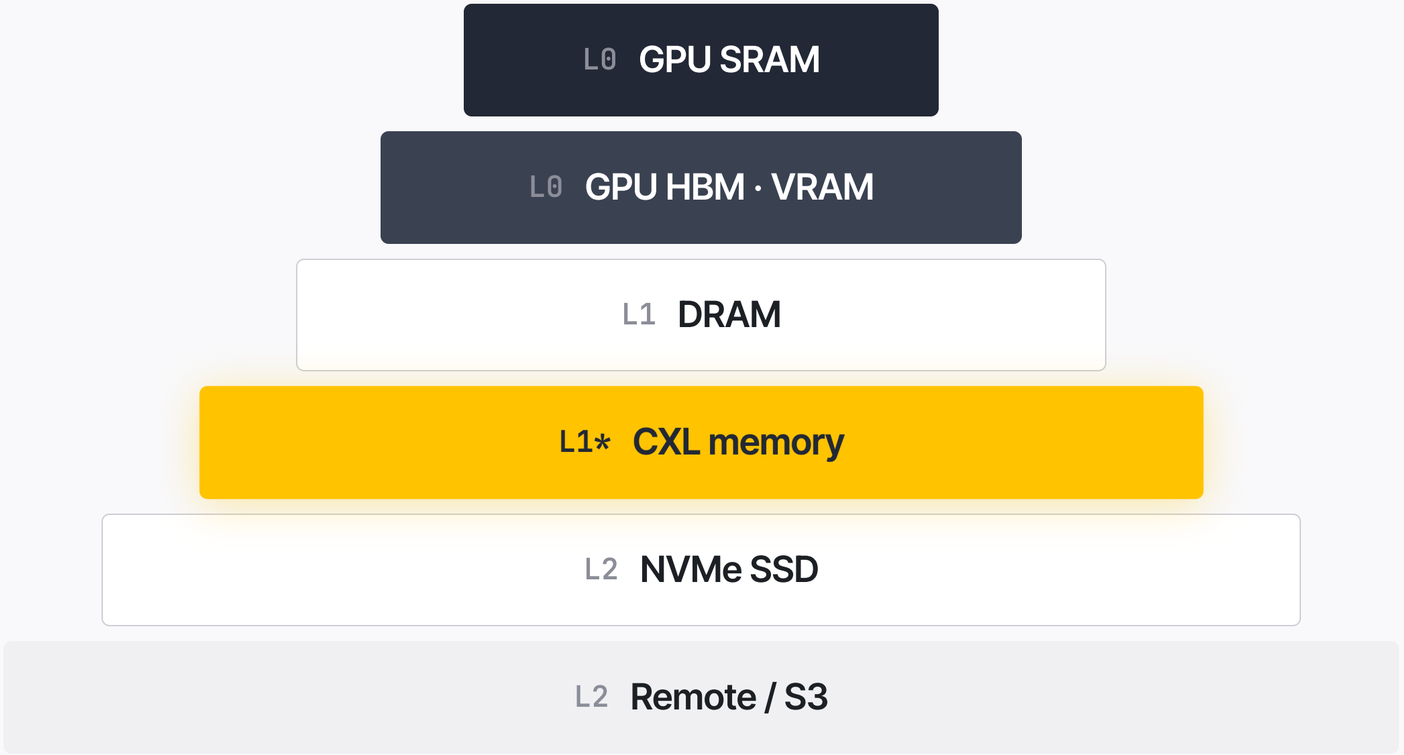
© 2026 XCENA Inc.

STORAGE HIERARCHY

The Memory Pyramid

↑ faster

smaller



↓ slower

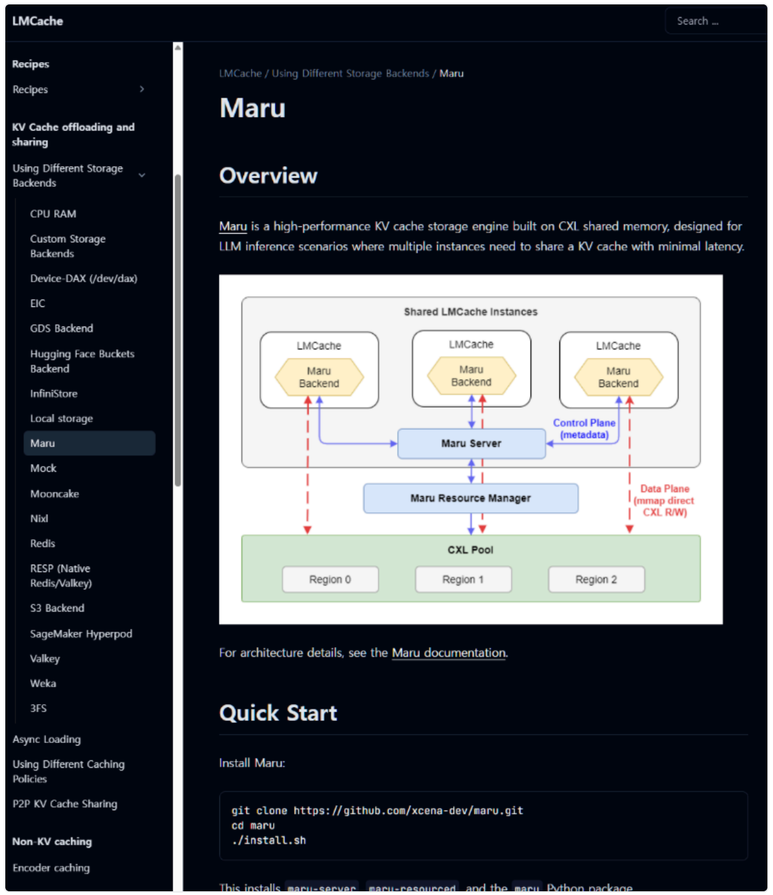
larger

© 2026 XCENA Inc.

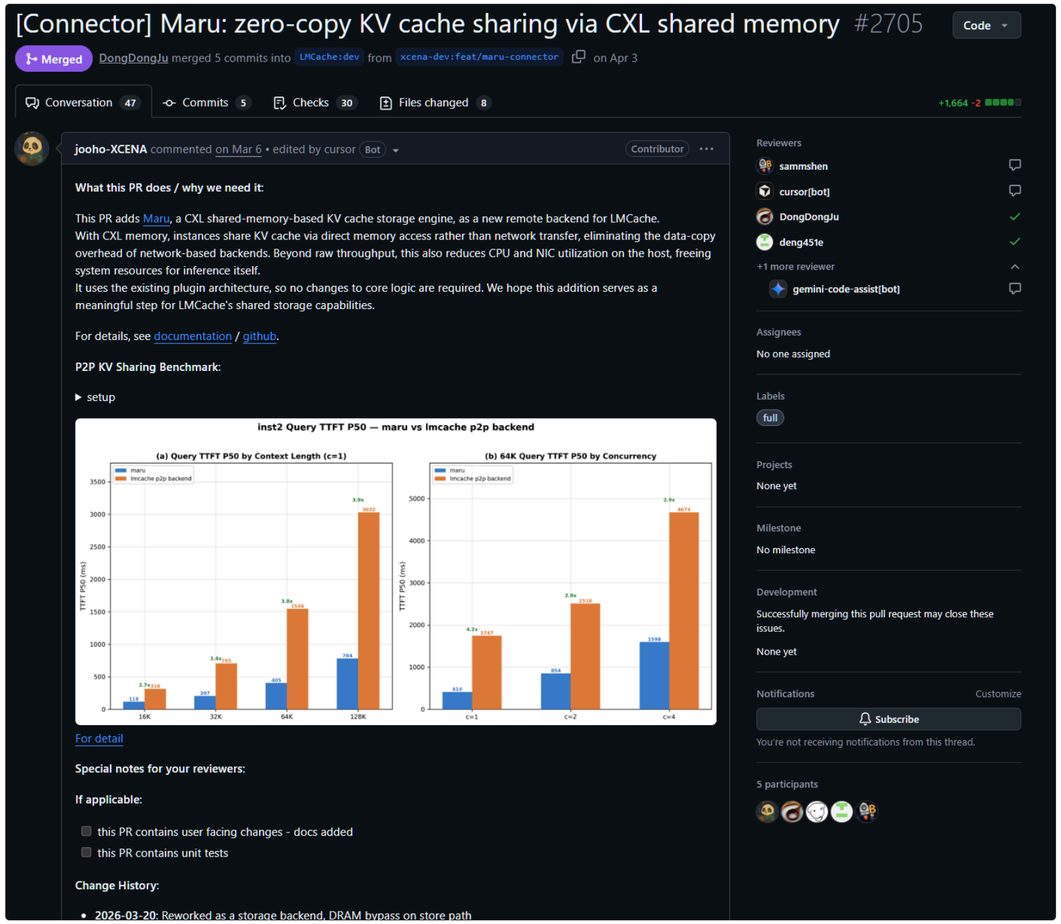
Maru — Zero-Copy KV Sharing over CXL

Share, don't move. Every instance reads one KV copy in the CXL pool by direct memory access — only metadata travels. No network transfer, no data-copy overhead, and it drops into LMCache as a plugin backend with no core changes.

Architecture · LMCache docs — instances → Maru Server (metadata) → Resource Manager → CXL pool



Merged · PR #2705 — TTFT P50 vs p2p: 2.7–3.9× by context, up to 4.2× by concurrency



Source: LMCache docs & PR #2705 (xcena-dev/feat/maru-connector → LMCache:dev). Benchmark vendor-reported.

© 2026 XCENA Inc.

WHAT'S NEXT

Hands-On with Tensormesh

> We covered inference and reuse, the LMCache architecture and data paths, and the module internals. Next we run the production build — Tensormesh — end to end.

1 · LLM Inference | 2 · Architecture & Data Paths | 3 · Module Internals | → Hands-on

© 2026 XCENA Inc.

XCENA

Xcelerate Your Intelligence

JOIN THE TEAM

We're hiring.

> We're building **Maru** — shared-memory KV cache on CXL. Come help make "don't move data, share the memory" the default for LLM inference.

Systems / Kernel

CXL & Firmware

LLM Inference & Serving

Analytics Engine

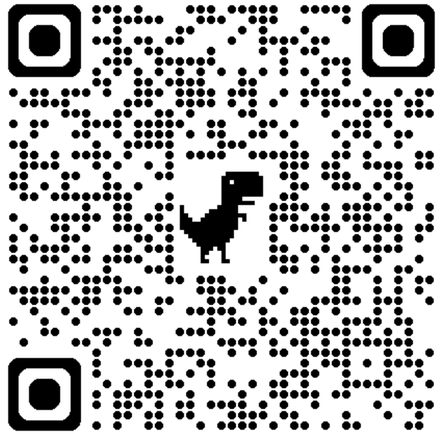
© 2026 XCENA Inc.

KCC 2026 · TUTORIAL

감사합니다.

Thank you

Inference · LMCache · CXL · Maru



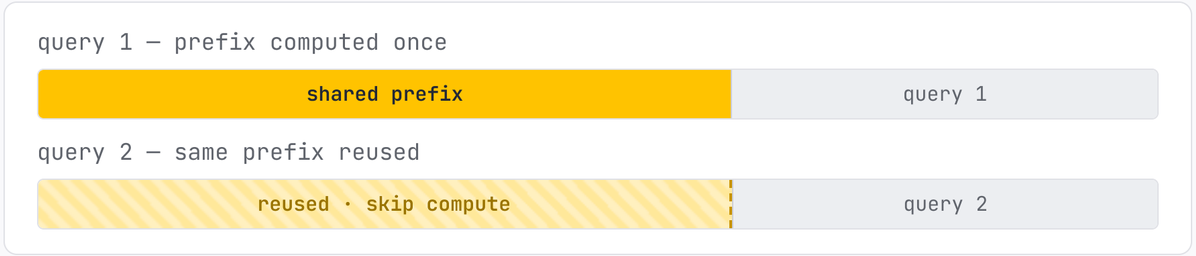
© 2026 XCENA Inc.

VLLM · THE PREFIX CACHE LMCACHE BUILDS ON

APC — Automatic Prefix Caching

Caches the KV of earlier queries, so a new query sharing the **same prefix** reuses that KV and **skips recomputing** the shared part.

`enable with enable_prefix_caching=True`



where APC helps most

Long-document query
same long doc, many different questions → process the doc once, reuse its KV.

Multi-round conversation
reuse the chat-history KV across every later turn of the session.

Limits — prefill only
- **Decode unchanged** little gain when answers are long (decode-bound).
- **Needs a shared prefix** no overlap with cached queries → nothing to reuse.

Source: vLLM docs — Automatic Prefix Caching. LMCache extends this same idea beyond HBM, across CPU / storage / CXL tiers.

© 2026 XCENA Inc.